

Table of Contents

Table of Contents	9
List of Figures	11
List of Tables	13
Abbreviations	14
Rezumat în limba română.....	15
1 Work Planning.....	23
2 State of the Art	24
3 Theoretical Fundamentals	27
3.1 Virtual Private Network.....	27
3.2 VPN tunneling	28
3.3 Virtual Network Interfaces.....	29
3.4 VPN Protocols – QUIC Protocol: Comparison with TCP and UDP, Architecture and Features	30
3.4.1 TCP protocol.....	30
3.4.2 UDP Protocol.....	32
3.4.3 QUIC protocol	32
3.4.4 Advantages of QUIC	36
3.4.5 Comparison between TCP and QUIC protocol	37
3.5 Network Address Translation (NAT).....	38
4 Implementation.....	39
4.1 Brief Presentation.....	39
4.2 Preparing the local environment	40
4.3 Installing and Configuring the Required Software Components for the VPN Solution	42
4.4 Initial QUIC Communication Setup.....	44
4.5 Integration of Virtual TUN Interfaces.....	45
4.6 Packet Transmission Through QUIC Streams on the Client Side	48
4.7 Reception and Reconstruction of QUIC Packets on the Server Side	49
4.8 IP Forwarding and NAT Configuration	50
4.9 Complete VPN Operation and Packet Flow.....	51
5 Experimental Results.....	52
5.1 Experimental Demonstration of the VPN Implementation	52
5.2 Comparison of QUIC and TCP Traffic.....	55
5.2.1 QUIC Traffic	55
5.2.2 TCP Traffic.....	57
5.2.3 Differences between QUIC and TCP.....	58

5.3	Comparison with Existing VPN Solutions.....	59
5.4	Impact of artificial network delay on VPN throughput	61
6	Conclusions	63
7	References	Error! Bookmark not defined.
8	Appendix	Error! Bookmark not defined.
8.1	Server Code.....	Error! Bookmark not defined.
8.2	Client Code	Error! Bookmark not defined.

List of Figures

Figure 1. Remote Access VPN example [Kha23]	27
Figure 2. Site-to-Site VPN example [Com23]	28
Figure 3. Main components of a VPN Tunnel	29
Figure 4. TCP Three-Way Handshake	31
Figure 5. Head-of-Line Blocking	32
Figure 6. UDP connection establishment [Gee25]	32
Figure 7. The structure of a QUIC packet with long header [Kum20]	33
Figure 8. The structure of a QUIC packet with short header [Tan24]	33
Figure 9. QUIC protocol components [Sal21]	34
Figure 10. TCP vs UDP Connections [Kum20]	34
Figure 11. Initial 1-RTT handshake process QUIC [Zha21]	35
Figure 12. Example of QUIC connection migration	37
Figure 13. Architecture of the implemented QUIC VPN solution	39
Figure 14. Main steps for creating the virtual machine in Oracle VM VirtualBox	40
Figure 15. Steps for attaching the Ubuntu ISO image in Oracle VM VirtualBox	41
Figure 16. Virtual Machine configured to a Bridged Adapter	42
Figure 17. Commands used for installing Python3	42
Figure 18. Commands used for the verification if installation	42
Figure 19. Commands used for installing the aioquic library	42
Figure 20. Verification of the aioquic library installation	43
Figure 21. Commands used for generating the certificate and private key	43
Figure 22. Commands used for creating Python files	43
Figure 23. Commands used for creating a directory	43
Figure 24. Command used for moving files	43
Figure 25. Initial Server and Client Codes	44
Figure 26. Initial implementation of the QUIC client-server connection	45
Figure 27. Configuration of the virtual TUN interfaces used by the VPN implementation	48
Figure 28. Terminal outputs illustrating the initialization of the QUIC VPN application	52
Figure 29. Background traffic captured on the server and client TUN interface before generating user traffic	53
Figure 30. ICMP packet transmission through the QUIC VPN tunnel and the corresponding terminal outputs	53
Figure 31. ICMP packets captured on the client-side TUN interface.	54
Figure 32. Ping statistics obtained through the QUIC VPN tunnel	55
Figure 33. Server output after the client disconnects	55
Figure 34. First packet exchanged between the client and the server	55
Figure 35. Header and payload fields of the first QUIC Initial packet.	56
Figure 36. CRYPTO frame carrying the TLS 1.3 Client Hello message.	56
Figure 37. QUIC Short Header packet captured after connection establishment	57
Figure 38. First TCP SYN packet captured during connection establishment	57
Figure 39. Main fields of the TCP SYN packet	58

Figure 40. Latency measured over multiple runs	59
Figure 41. Latency Comparison between QUIC VPN and OpenVPN.....	60
Figure 42. Throughput measurements under moderate network delays.....	62

List of Tables

Table 1. Comparison between TCP and QUIC	37
Table 2. Quic and TCP traffic comparison.....	58
Table 3. Differences between the implemented VPN solution and the widely used VPN technologies (WireGuard and OpenVPN)	60

Abbreviations

ACK (Acknowledgment)
AES (Advanced Encryption Standard)
ARPANET (Advanced Research Projects Agency Network)
CRYPTO (Cryptographic Frame)
HOL (Head-of-Line)
HTTP (Hypertext Transfer Protocol)
HTTP/1.1 (Hypertext Transfer Protocol Version 1.1)
HTTP/3 (Hypertext Transfer Protocol Version 3)
ICMP (Internet Control Message Protocol)
IETF (Internet Engineering Task Force)
IKEv2 (Internet Key Exchange Version 2)
IP (Internet Protocol)
IPsec (Internet Protocol Security)
IPv4 (Internet Protocol Version 4)
IPv6 (Internet Protocol Version 6)
ISO (International Organization for Standardization)
L2TP (Layer 2 Tunneling Protocol)
MSS (Maximum Segment Size)
NAT (Network Address Translation)
OpenVPN (Open Virtual Private Network)
OS (Operating System)
PFS (Perfect Forward Secrecy)
PPTP (Point-to-Point Tunneling Protocol)
QUIC (Quick UDP Internet Connections)
RFC (Request for Comments)
RTT (Round-Trip Time)
SACK (Selective Acknowledgment)
SSL (Secure Sockets Layer)
SYN (Synchronize)
SYN-ACK (Synchronize-Acknowledgment)
TAP (Terminal Access Point)
TCP (Transmission Control Protocol)
TLS (Transport Layer Security)
TLS 1.3 (Transport Layer Security Version 1.3)
TUN (Network TUNnel)
UDP (User Datagram Protocol)
VM (Virtual Machine)
VPN (Virtual Private Network)
Wi-Fi (Wireless Fidelity)

Rezumat în limba română

Introducere

În ultimii ani, securitatea comunicațiilor prin internet a devenit o preocupare majoră atât pentru utilizatorii individuali, cât și pentru organizații. Creșterea numărului de dispozitive conectate și dezvoltarea serviciilor online au dus la necesitatea unor mecanisme capabile să asigure confidențialitatea, integritatea și protecția datelor transmise prin rețele publice. Una dintre cele mai utilizate soluții pentru atingerea acestui obiectiv este reprezentată de rețelele private virtuale (Virtual Private Networks), care permit realizarea unor conexiuni securizate între două sau mai multe puncte aflate la distanță.

Majoritatea implementărilor VPN existente utilizează protocoale consacrate precum OpenVPN, WireGuard sau IKEv2/IPsec. Aceste soluții oferă un nivel ridicat de securitate, însă fiecare prezintă anumite limitări legate de latență, complexitatea configurării sau mecanismele utilizate pentru stabilirea conexiunii. În acest context, protocolul QUIC reprezintă o alternativă modernă, proiectată pentru a reduce timpul necesar inițializării comunicației și pentru a integra mecanismele de securitate direct în protocolul de transport.

Lucrarea are ca obiectiv dezvoltarea unei implementări experimentale a unui VPN bazat pe protocolul QUIC și pe interfețele virtuale TUN din sistemul de operare Linux. Soluția propusă urmărește transportul pachetelor IP printr-un tunel securizat realizat peste protocolul QUIC, utilizând o arhitectură client-server implementată în limbajul Python cu ajutorul bibliotecii aioquic. Pentru validarea implementării au fost utilizate două mașini virtuale Ubuntu, iar funcționarea sistemului a fost verificată prin transmiterea de trafic IP și analiza pachetelor capturate cu Wireshark. Pe lângă prezentarea implementării, lucrarea include o comparație între traficul generat de protocolul QUIC și cel al unei conexiuni TCP clasice, evidențiind diferențele dintre cele două protocoale în ceea ce privește stabilirea conexiunii, structura pachetelor și mecanismele utilizate pentru securizarea comunicației.

Lucrarea este structurată în mai multe capitole care urmăresc atât prezentarea aspectelor teoretice, cât și implementarea practică a soluției. În prima parte sunt prezentate noțiunile fundamentale referitoare la VPN-uri, tunelare, interfețele virtuale TUN și protocolul QUIC, împreună cu principalele caracteristici care au stat la baza alegerii acestuia. În continuare este descris procesul de implementare al aplicației, de la configurarea mediului de dezvoltare până la realizarea comunicației prin tunelul VPN. Ultima parte prezintă rezultatele experimentale și analiza traficului generat, precum și comparația dintre protocolul QUIC și o conexiune TCP.

Fundamentare teoretică

O rețea privată virtuală reprezintă o tehnologie care permite realizarea unei comunicații securizate prin intermediul unei rețele publice, cum este Internetul. În locul utilizării unei conexiuni fizice dedicate între două echipamente, un VPN creează un tunel virtual prin care datele sunt transmise în mod securizat. În interiorul acestui tunel, informațiile sunt protejate prin criptare, reducând posibilitatea interceptării sau modificării lor de către terți.

Un avantaj important al utilizării unui VPN îl reprezintă protejarea identității utilizatorului prin mascarea adresei IP publice. În plus, această tehnologie permite accesarea resurselor unei rețele private de la distanță și oferă posibilitatea utilizării unor servicii care pot fi restricționate

geografic. Din aceste motive, VPN-urile sunt utilizate atât în mediul organizațional, cât și de utilizatorii individuali preocupați de confidențialitatea comunicațiilor. Conform modului de utilizare, VPN-urile pot fi împărțite în două categorii principale: Remote Access VPN și Site-to-Site VPN. Un Remote Access VPN permite unui utilizator aflat într-o locație externă să stabilească o conexiune securizată către o rețea privată, având acces la resursele acesteia ca și cum s-ar afla în rețeaua locală. Acest tip de conexiune este utilizat frecvent de angajații care lucrează de la distanță și reprezintă arhitectura utilizată și în cadrul implementării prezentate în această lucrare.

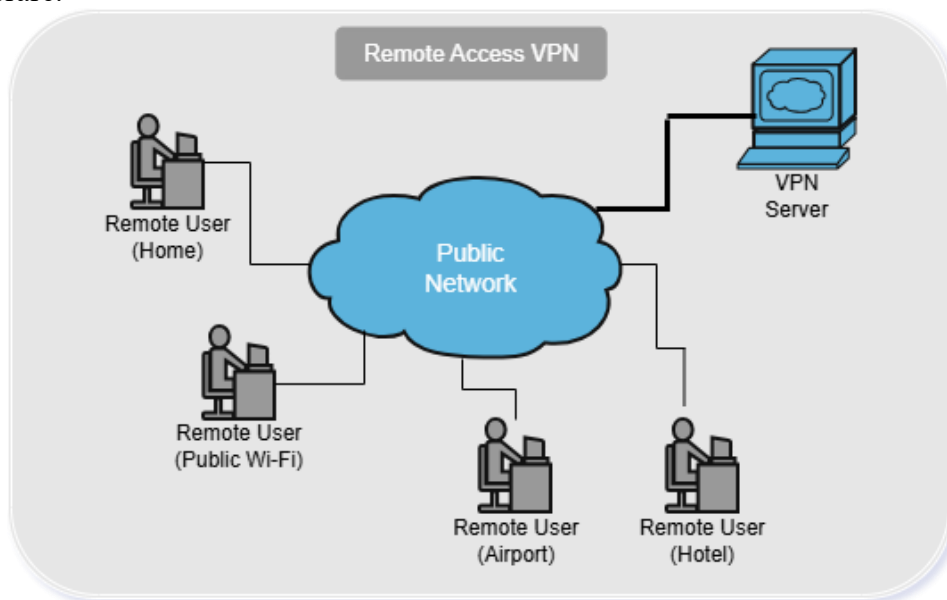


Figura 1 – Remote Access VPN example

În schimb, un Site-to-Site VPN conectează două sau mai multe rețele aflate în locații diferite. În această situație nu sunt conectați utilizatori individuali, ci întregi segmente de rețea, permițând schimbul securizat de informații între sedii diferite ale aceleiași organizații sau între organizații distincte.

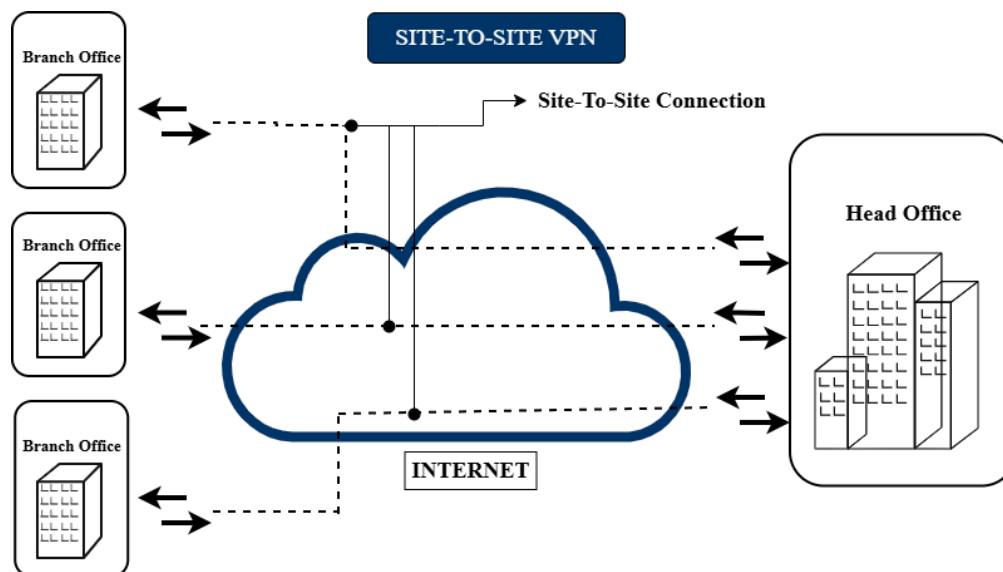


Figure 2 – Site-to-Site VPN example

Indiferent de arhitectura utilizată, funcționarea unui VPN se bazează pe existența unui tunel virtual prin care traficul este transportat în siguranță. În funcție de cerințele aplicației, implementările VPN pot include și alte componente, precum firewall-uri sau servere proxy, care contribuie la controlul accesului și la protejarea comunicației. Principiul fundamental al unui VPN este procesul de tunelare. În timpul transmiterii informațiilor prin internet, pachetele pot fi interceptate sau analizate de alte dispozitive aflate pe traseul comunicației. Pentru a preveni acest lucru, pachetele originale nu sunt transmise direct, ci sunt încapsulate într-un nou pachet și transportate printr-un tunel virtual.

Înainte de transmiterea datelor, cele două capete ale comunicației negociază parametri necesari securizării conexiunii. După stabilirea acestora, pachetul IP original este criptat și încapsulat într-un nou pachet care conține informațiile necesare transportului prin rețeaua publică. La destinație, operațiile sunt realizate în ordine inversă: pachetul este decapsulat, decriptat și livrat către aplicația sau dispozitivul corespunzător.

Încapsularea presupune introducerea unui pachet în interiorul altuia. Antetul exterior conține informațiile necesare rutării prin Internet, în timp ce antetul original este păstrat în interiorul tunelului și devine disponibil doar după eliminarea învelișului exterior. Acest mecanism permite transportul transparent al traficului IP prin rețele publice fără modificarea informațiilor originale.

Securitatea comunicației este asigurată prin utilizarea algoritmilor criptografici și a mecanismelor de autentificare, care verifică identitatea participanților înainte de începerea schimbului de date. Astfel, doar dispozitivele autorizate pot transmite și primi informații prin tunelul VPN. După încapsulare, pachetul este criptat înainte de a fi transmis prin rețeaua publică. Criptarea transformă informațiile într-o formă care poate fi interpretată doar de dispozitivele care dețin cheile criptografice corespunzătoare. Înainte de schimbul efectiv de date sunt utilizate și mecanisme de autentificare, care verifică identitatea participanților la comunicație. Prin combinarea încapsulării, criptării și autentificării, tunelul VPN oferă un nivel ridicat de confidențialitate și protecție a datelor transmise.

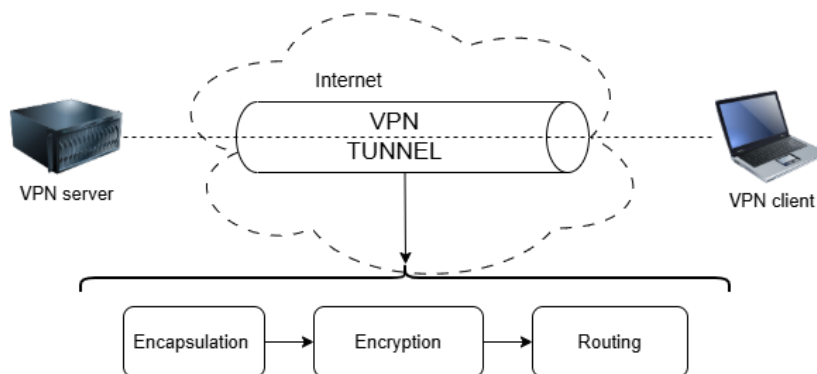


Figura 3 – Main components of a VPN Tunnel

Implementarea unui VPN necesită existența unei interfețe prin care aplicația poate interacționa cu stiva de rețea a sistemului de operare. În sistemele Linux, această funcționalitate este oferită de interfețele virtuale TUN și TAP. Interfața TUN funcționează la nivelul rețelei (Layer 3) și este destinată exclusiv transportului pachetelor IP. Spre deosebire de o interfață fizică, aceasta este creată software de nucleul sistemului de operare și permite aplicațiilor din spațiul utilizatorului să citească și să scrie direct pachete IP. Din acest motiv, interfața TUN este utilizată pe scară largă în implementările VPN, deoarece oferă posibilitatea interceptării și redirectionării traficului înainte ca acesta să fie transmis prin rețeaua fizică.

Comparativ cu interfața TAP, care operează la nivelul legăturii de date și transmite cadre Ethernet, TUN procesează exclusiv pachete IP. Această caracteristică o face mai potrivită pentru implementarea unui VPN de tip Layer 3.

TCP este un protocol orientat pe conexiune care asigură transmiterea fiabilă a datelor prin mecanisme de confirmare, retransmisie și control al erorilor. Înainte de transmiterea datelor, conexiunea este stabilită prin mecanismul Three-Way Handshake, ceea ce introduce o latență suplimentară. O limitare importantă a protocolului TCP este fenomenul Head-of-Line Blocking, prin care pierderea unui pachet întârzie procesarea pachetelor următoare, chiar dacă acestea au fost deja recepționate. Acest comportament poate afecta performanța aplicațiilor care transmit mai multe fluxuri de date simultan. UDP elimină necesitatea stabilirii unei conexiuni și reduce latența, însă nu oferă mecanisme de fiabilitate. Pornind de la avantajele UDP, protocolul QUIC integrează mecanisme de securitate și fiabilitate, oferind stabilirea rapidă a conexiunii, multiplexarea fluxurilor și reducerea efectelor produse de Head-of-Line Blocking.

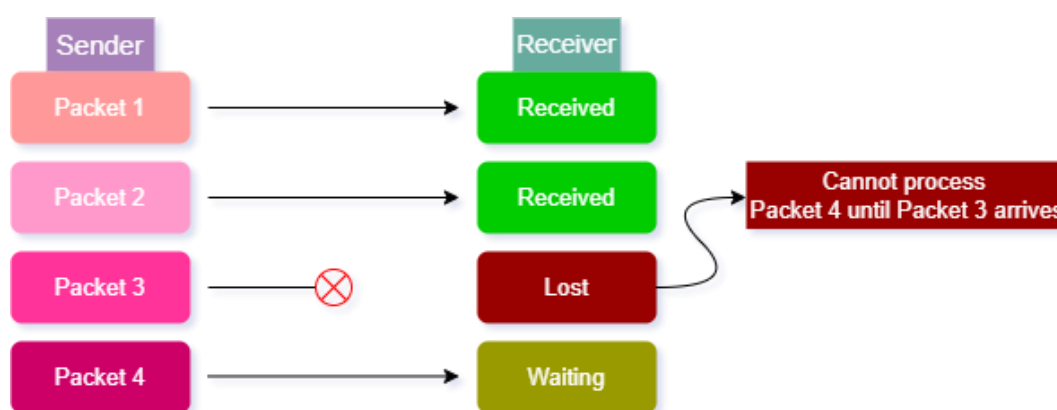


Figura 4 – Head-of-Line Blocking

QUIC este un protocol de transport care funcționează peste UDP și a fost dezvoltat pentru a reduce latența și a îmbunătăți performanța comunicațiilor moderne. Spre deosebire de TCP, acesta integrează mecanismele de securitate direct în protocol, utilizând TLS 1.3 pentru autentificare și criptarea datelor. În prezent, QUIC reprezintă baza protocolului HTTP/3. Protocolul utilizează două tipuri de antet: Long Header și Short Header. Long Header este folosit în etapa de stabilire a conexiunii și conține informațiile necesare negocierii parametrilor comunicației, precum versiunea protocolului, Connection ID și numărul pachetului. După finalizarea handshake-ului, QUIC utilizează formatul Short Header, care conține doar informațiile necesare transmiterii datelor și ajută la reducerea overhead-ului protocolului. Datele sunt transportate în payload-ul pachetului, payload care este protejat prin criptare după stabilirea conexiunii.

Unul dintre principalele avantaje ale protocolului QUIC este timpul redus de stabilire a conexiunii, prin utilizarea mecanismelor 1-RTT și 0-RTT. De asemenea, protocolul permite transmiterea simultană a mai multor fluxuri de date independente, reducând efectele fenomenului Head-of-Line Blocking. Un alt avantaj este connection migration, realizată prin intermediul Connection ID-ului, care permite menținerea conexiunii chiar dacă se modifică adresa IP sau rețeaua utilizată de client. Comparativ cu TCP, QUIC combină mecanismele de transport și securitate într-un singur protocol și elimină necesitatea utilizării unui protocol suplimentar pentru criptarea comunicației.

Implementare

Capitolul de implementare prezintă etapele urmate pentru dezvoltarea unei soluții VPN bazate pe protocolul QUIC și pe interfețele virtuale TUN. Implementarea utilizează o arhitectură client–server, în care clientul preia pachetele IP generate de sistemul de operare și le transmite către server prin intermediul unei conexiuni QUIC. La destinație, pachetele sunt reconstruite și reinjectate în stiva de rețea, permițând schimbul de trafic prin tunelul virtual.

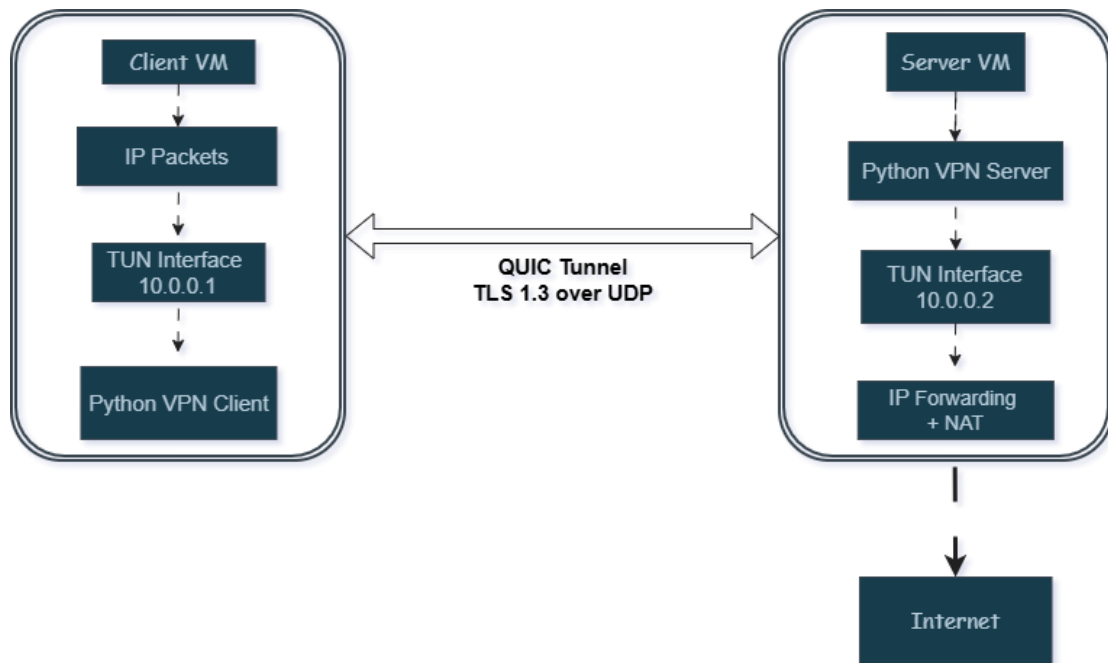


Figura 5 – Architecture of the implemented QUIC VPN solution

Pentru dezvoltarea și testarea aplicației a fost creat un mediu local bazat pe Oracle VM VirtualBox, în care au fost configurate două mașini virtuale Ubuntu, una având rolul de client, iar cealaltă rolul de server. Utilizarea mașinilor virtuale a permis simularea unei comunicații reale între două sisteme independente și testarea funcționării soluției într-un mediu controlat. După crearea mașinilor virtuale și atașarea imaginii ISO Ubuntu, acestea au fost configurate pentru utilizarea unui adaptor de rețea de tip Bridged, astfel încât fiecare mașină virtuală să poată comunica direct în rețea.

După pregătirea mediului de dezvoltare au fost instalate componentele software necesare implementării aplicației. Soluția a fost dezvoltată în limbajul Python, utilizând biblioteca aioquic pentru implementarea protocolului QUIC. De asemenea, au fost generate certificatele necesare stabilirii conexiunilor securizate și au fost instalate certificatele necesare pentru configurarea interfețelor de rețea și analizarea traficului. Prima etapă a implementării a constat în realizarea unei comunicații elementare între client și server prin protocolul QUIC. În această etapă au fost implementate mecanismele necesare inițializării conexiunii și schimbului de mesaje între cele două aplicații, fără utilizarea interfețelor virtuale. Această implementare a permis verificarea funcționării protocolului QUIC înainte de integrarea mecanismelor specifice unui VPN. Ulterior au fost integrate interfețele virtuale TUN atât pe client, cât și pe server. Fiecare aplicație creează și configurează propria interfață virtuală, atribuindu-i adresa IP corespunzătoare. După configurare, sistemul de operare poate transmite și primi pachete IP prin intermediul acestor interfețe, aplicațiile având posibilitatea de a citi și scrie direct traficul care le traversează.

După configurarea interfețelor TUN, clientul monitorizează permanent interfața virtuală și citește pachetele IP generate de sistemul de operare. Acestea sunt încapsulate și transmise prin fluxurile QUIC către server. Utilizarea fluxurilor QUIC permite transportul datelor prin conexiunea securizată dintre cele două aplicații. La primirea datelor, serverul extrage conținutul transmis prin fluxul QUIC și reconstruiește pachetul IP original. Acesta este apoi scris în interfața TUN a serverului, astfel încât sistemul de operare să îl proceseze ca pe un pachet primit printr-o interfață de rețea obișnuită. În acest mod este realizată comunicarea dintre cele două capete ale tunelului virtual.

Pentru a permite transmiterea traficului către alte rețele, implementarea include configurarea mecanismelor IP Forwarding și NAT. Activarea IP Forwarding permite sistemului de operare să redirecționeze pachetele între interfața TUN și interfața fizică de rețea, iar regulile NAT realizează translatarea adreselor înainte ca pachetele să fie transmise către rețeaua externă. Funcționarea completă a aplicației începe prin pornirea serverului, care rămâne în așteptarea conexiunilor venite de la client. După inițializarea clientului și stabilirea conexiunii QUIC, sunt create interfețele TUN pe ambele sisteme.

În momentul în care utilizatorul generează trafic, de exemplu prin trimiterea unui pachet ICMP, pachetul este preluat din interfața TUN a clientului, transmis prin conexiunea QUIC, reconstruit pe server și reinjectat în interfața TUN corespunzătoare. Configurarea IP Forwarding și NAT permite apoi direcționarea pachetului către destinația finală. Răspunsul urmează același traseu în sens invers, fiind transmis prin tunelul QUIC până la client.

Rezultate experimentale

Capitolul dedicat rezultatelor experimentale prezintă testele realizate pentru verificarea funcționării soluției VPN dezvoltate și analiza traficului generat în timpul comunicației. Scopul experimentelor a fost validarea transmiterii pachetelor prin tunelul QUIC și evidențierea diferențelor dintre protocolul QUIC și o conexiune TCP tradițională.

În prima etapă a experimentelor au fost pornite aplicațiile client și server, iar terminalele au confirmat inițializarea cu succes a conexiunii și configurarea interfețelor virtuale TUN. Înainte de generarea traficului utilizatorului, pe ambele interfețe TUN a fost observat doar trafic de fundal specific sistemului de operare, ceea ce a confirmat funcționarea corectă a interfețelor virtuale.

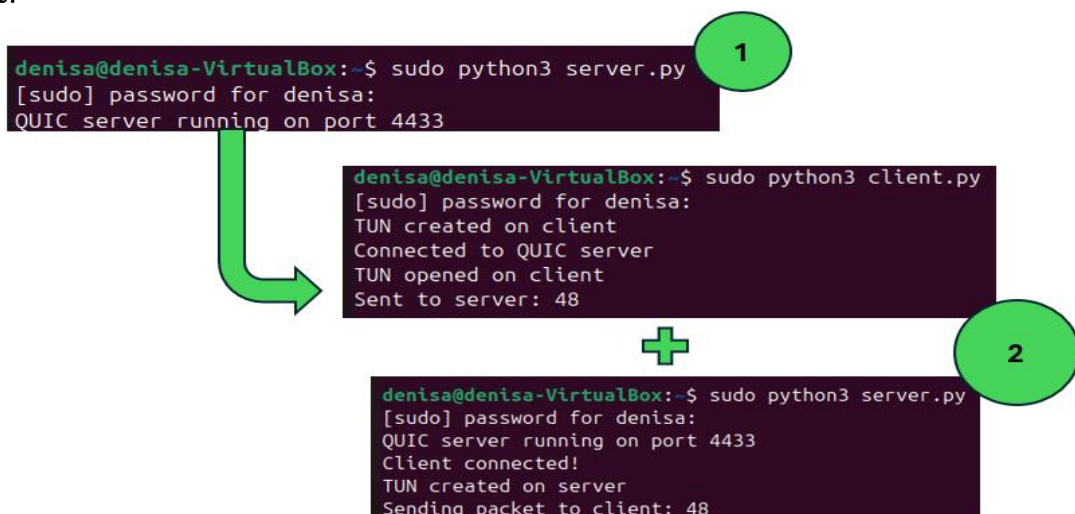


Figura 7 – Terminal outputs illustrating the initialization of the QUIC VPN application

Pentru verificarea funcționării tunelului VPN au fost transmise pachete ICMP între cele două mașini virtuale utilizând comanda ping. Pachetele generate de client au fost preluate prin interfața TUN, transmise prin conexiunea QUIC și reconstruite pe server, unde au fost reinjectate în stiva de rețea. Capturile realizate au confirmat că pachetele au traversat tunelul VPN fără modificări, iar răspunsurile au fost recepționate cu succes de client. Statisticile obținute în urma testului au demonstrat transmiterea corectă a pachetelor și funcționarea completă a implementării.

```
^C
--- 10.0.0.2 ping statistics ---
451 packets transmitted, 451 received, 0% packet loss, time 450690ms
rtt min/avg/max/mdev = 2.123/14.050/36.141/4.954 ms
```

Figura 8 – Ping statistics obtained through the QUIC VPN tunnel

După validarea funcționării VPN-ului, traficul generat de aplicație a fost analizat cu ajutorul aplicației Wireshark și comparat cu traficul unei aplicații simple bazate pe protocolul TCP. În cazul protocolului QUIC, analiza a evidențiat procesul de stabilire a conexiunii, schimbul inițial de pachete și integrarea mesajelor TLS 1.3 în cadrul protocolului. Capturile au ilustrat atât pachetele QUIC Initial utilizate în timpul inițializării conexiunii, cât și pachetele Short Header utilizate după finalizarea handshake-ului. În cazul conexiunii TCP, analiza a evidențiat mecanismul tradițional Three-Way Handshake, format din schimbul pachetelor SYN, SYN-ACK și ACK necesare stabilirii conexiunii înainte de transmiterea datelor. QUIC integrează mecanismele de transport și securitate într-un singur protocol, în timp ce TCP necesită utilizarea unui protocol suplimentar, precum TLS, pentru asigurarea comunicației securizate.

Pe baza capturilor realizate a fost efectuată și o comparație între cele două protocoale. Analiza a evidențiat diferențe privind modul de stabilire a conexiunii, structura pachetelor și integrarea mecanismelor de securitate. În timp ce TCP utilizează un handshake separat și necesită un protocol suplimentar pentru criptare, QUIC include criptarea direct în protocol și permite schimbul mai rapid al datelor după inițializarea conexiunii.

```
▼ QUIC IETF
  ► QUIC Connection information
    [Packet Length: 491]
    1... .. = Header Form: Long Header (1)
    .1.. .. = Fixed Bit: True
    ..00 .. = Packet Type: Initial (0)
    [.... 00.. = Reserved: 0]
    [.... ..01 = Packet Number Length: 2 bytes (1)]
    Version: 1 (0x00000001)
    Destination Connection ID Length: 8
    Destination Connection ID: e4ee68be24417b95
    Source Connection ID Length: 8
    Source Connection ID: 4835c79513274f3a
    Token Length: 0
    Length: 465
    [Packet Number: 0]
    Payload [truncated]: cccf7bc8f104470d6db9f47155959e53ff5ada16934f723d67
```

Figura 9 - Header and payload fields of the first QUIC Initial packet

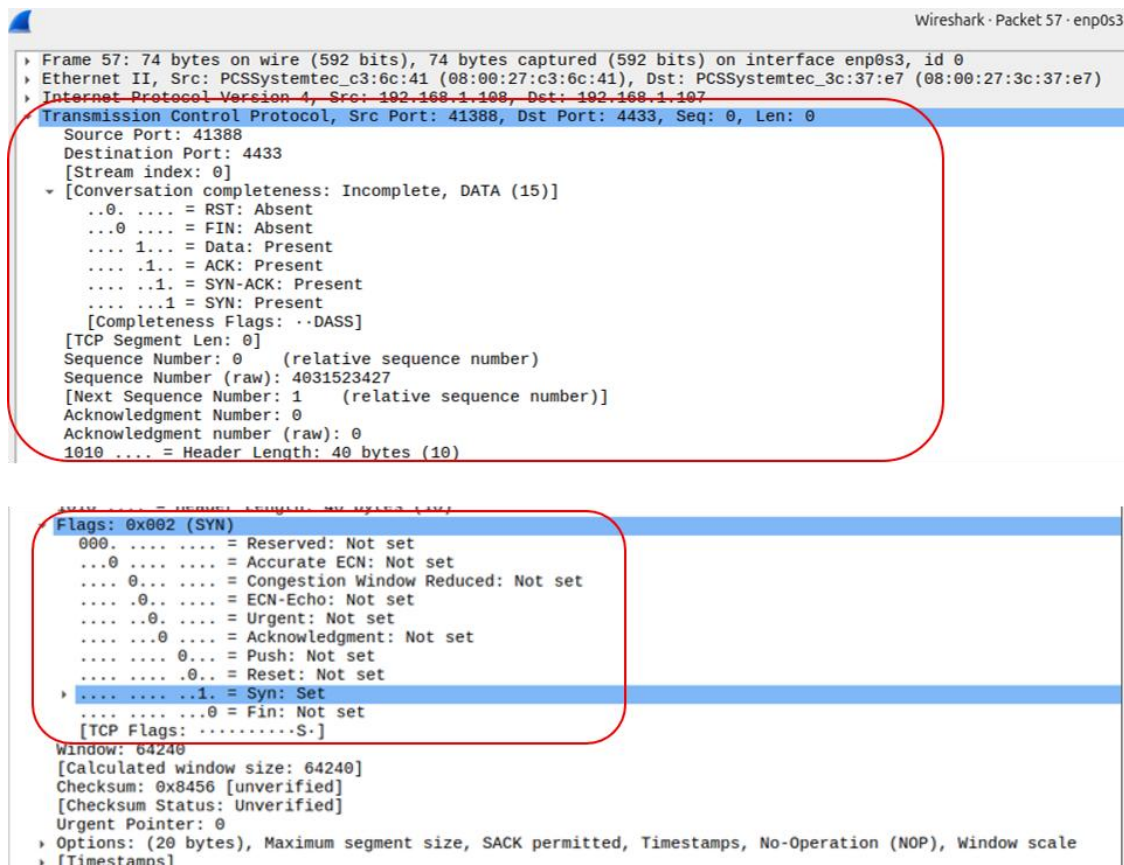


Figura 10 - Main fields of the TCP SYN packet

Implementarea realizată a fost comparată totodată și cu două dintre cele mai utilizate tehnologii VPN, WireGuard și OpenVPN. Pe lângă comparația arhitecturală, a fost evaluată și latența de stabilire a conexiunii, prin compararea implementării propuse cu OpenVPN. De asemenea, a fost analizat impactul întârzierilor artificiale asupra throughput-ului soluției dezvoltate, pentru a observa comportamentul acesteia în diferite condiții de rețea. Deși soluția dezvoltată are un caracter experimental și nu urmărește să înlocuiască implementările consacrate, aceasta demonstrează că protocolul QUIC poate fi utilizat pentru realizarea unui tunel VPN funcțional și oferă o bază pentru dezvoltări viitoare.

Concluzie

Lucrarea a urmărit dezvoltarea unei soluții VPN experimentale bazate pe protocolul QUIC și pe interfețele virtuale TUN, utilizând o arhitectură client-server implementată în Python. Implementarea și testele realizate au demonstrat funcționarea corectă a tunelului VPN, precum și transmiterea cu succes a pachetelor IP între cele două sisteme. Compararea traficului QUIC și TCP a evidențiat diferențele dintre cele două protocoale și modul în care QUIC integrează mecanismele de transport și securitate în cadrul aceleiași conexiuni. Astfel, implementarea realizată confirmă posibilitatea utilizării protocolului QUIC pentru dezvoltarea unei soluții VPN funcționale și constituie un punct de plecare pentru extinderea și evaluarea acesteia în scenarii mai complexe.

1 Work Planning

Task Name	Duration	Start	Finish
Choosing the subject	6 days	2 November 2025	8 November 2025
Theoretical Research	35 days	8 November 2025	12 December 2025
State of the Art	6 days	12 December 2025	18 December 2025
Setting up the development environment	4 days	6 March 2026	9 March 2026
Initial QUIC client-server implementation	20 days	10 March 2026	30 March 2026
Integration of the TUN interfaces	18 days	30 March 2026	16 April 2026
Packet forwarding and NAT configuration	12 days	17 April 2026	28 April 2026
Traffic analysis using Wireshark	10 days	29 April 2026	8 May 2026
Experimental evaluation	11 days	9 May 2026	20 May 2026
Writing and finalizing the diploma thesis	35 days	21 May 2026	26 June 2026

2 State of the Art

A VPN, or virtual private network, creates a digital link between the computer and a server belonging to a VPN provider, forming a point-to-point tunnel that encrypts personal information, conceals IP address, and enables to bypass internet website restrictions and firewalls. This guarantees that online activities remain confidential, safeguarded, and increasingly secure. Unencrypted data transmitted over a network can potentially be intercepted and accessed by unauthorized entities.

According to [Pal24a], the origins of VPN technology originated during the early stages of internet development. During the 1960s and 1970s, ARPANET (Advanced Research Projects Agency Network), the first large-scale computer network developed by the U.S. Department of Defense and a precursor to the modern Internet, connected remote computers through packet-switching networks, which created the need for secure communication across different networks. The introduction of the TCP/IP protocol in the 1980s enabled modern internet connectivity. In the mid-1990s, early VPN technologies emerged, including protocols such as PPTP and L2TP, which allowed users to establish encrypted tunnels and securely access remote networks via the Internet.

Palo Alto Networks [Pal24b] identifies OpenVPN, IKEv2/IPsec, WireGuard and L2TP/IPsec among the most used VPN protocols today. These protocols provide robust encryption and reliable connectivity, making VPNs widely used by both companies and individual users. The following section briefly describes some of the main protocols currently used in VPN implementations.

OpenVPN is a reliable, secure VPN protocol preferred in corporate settings for its high-level encryption and flexibility, as described in [Pal24b]. This protocol operates by establishing secure connections either point-to-point or site-to-site in routed or bridged setups. OpenVPN relies on the SSL/TLS protocol for authentication and key exchange.

OpenVPN functions by wrapping data in SSL/TLS encrypted packets, which are subsequently sent over the network. This protocol can operate over either the TCP (Transmission Control Protocol) or UDP (User Datagram Protocol) internet protocols. TCP is a connection-oriented protocol that guarantees dependable data packet delivery, whereas UDP is a lightweight connectionless protocol that focuses on speed, allowing OpenVPN to adapt to various network performance requirements.

[Pal24b] also states that companies use OpenVPN because it works well with different network types and can establish secure and reliable connections over both intranet and internet. OpenVPN provides robust security, an open-source structure that facilitates peer evaluation and audits, and the capability to bypass firewalls. However, it can be complex to configure, especially for users without advanced networking knowledge, and it is not natively supported by most operating systems.

Donenfeld [Don17] introduces WireGuard as a contemporary VPN protocol designed to improve performance while simplifying configuration and deployment. It aims to be quicker and more straightforward than older protocols, employing contemporary cryptographic techniques and an optimized open-source framework that enhances efficiency and user-friendliness. Unlike some traditional VPN protocols, WireGuard operates exclusively over UDP, which allows it to minimize latency and reduce protocol overhead.

The WireGuard paper [Don17] further describes a mechanism known as cryptokey routing, which creates secure VPN connections by associating network traffic with cryptographic identities. This approach simplifies configuration, reduces latency, and improves performance compared to traditional VPN protocols.

The paper [Pal24b] highlights that WireGuard's lightweight design allows it to run on a wide range of devices, from embedded systems to full servers, making it suitable for environments such as mobile and cloud-based deployments.

However, WireGuard comes with its limitations. The allocation of static IP addresses may lead to privacy concerns since it may facilitate user tracking. Furthermore, as a relatively new protocol, WireGuard has not yet undergone the same level of long-term testing and adoption as older VPN technologies.

RFC 7296 [Kau14] defines IKEv2/IPsec as a VPN solution that combines two components: IKEv2 (Internet Key Exchange version 2) and IPsec (Internet Protocol Security). IKEv2 is responsible for negotiating the connection, authenticating the communicating parties, and managing cryptographic keys, while IPsec provides encryption and secure transmission of data through an encrypted tunnel. This combination enables secure VPN connections with strong encryption and authentication mechanisms.

Symlex VPN [Sym24a] highlights IKEv2/IPsec as a protocol known for its high security and efficient performance. It supports strong encryption standards such as AES-256, provides protection mechanisms like Perfect Forward Secrecy, and can quickly re-establish VPN connections when the network changes (for example, when switching from Wi-Fi to mobile data).

[Sym24b] also notes several limitations of IKEv2/IPsec. The protocol typically uses UDP ports (such as 500 and 4500), which may be blocked by certain firewalls or restrictive network environments. Additionally, the configuration and deployment of IKEv2/IPsec can be relatively complex compared to some current VPN protocols, requiring more advanced networking knowledge.

Several limitations of the VPN protocols mentioned above, such as connection establishment latency, protocol complexity, and potential blocking in restrictive network environments, have motivated the exploration of alternative transport solutions. In this context, this work explores the use of the QUIC (Quick UDP Internet Connections) protocol as a potential approach to address some of these limitations, as QUIC is a transport protocol that was designed to improve performance, reduce latency, and enhance reliability in modern network communications.

Although QUIC provides several advantages, it was originally designed to optimize web traffic rather than VPN tunneling applications. As described in [Clo25], unlike traditional protocols such as TCP, which have been used for decades, QUIC was developed to reduce latency while enhancing reliability and security.

QUIC operates over UDP, a lightweight transport protocol, and incorporates encryption directly into its design. This integrated approach allows connections to be established more quickly while ensuring a high level of security by default, as explained in [Clo25]. Today, QUIC forms the basis of HTTP/3, the most recent version of the Hypertext Transfer Protocol used on the web.

The QUIC transport specification presented by Iyengar and Thomson [Iye21] presents several key features that distinguish QUIC from traditional transport protocols. One of the main advantages of QUIC is its reduced connection establishment latency. Traditional connections using TCP and TLS may require several round-trip times (RTTs) before secure communication can begin. QUIC reduces this overhead by enabling connection establishment in one RTT for new connections and even zero RTT for previously established connections, allowing data to be transmitted much faster. The authors also describe QUIC's multiplexing capabilities, which allow multiple data streams to be transmitted over a single connection. Unlike TCP, where packet loss in one stream can block the delivery of other streams, QUIC handles streams independently. This eliminates the head-of-line blocking problem, which can significantly improve performance in environments where packet loss occurs.

Another important feature discussed in [Iye21] is connection migration. While TCP connections are identified using a combination of parameters such as IP addresses and ports, QUIC uses a unique connection identifier. This allows connections to remain active even if the underlying network parameters change, for example when a device switches from Wi-Fi to mobile data. Furthermore, the paper highlights that QUIC integrates security directly into the protocol by incorporating contemporary encryption mechanisms based on TLS. Unlike TCP-based connections where encryption is added as an additional layer, QUIC performs authentication and cryptographic negotiation as part of its own handshake process.

Some modern proxy frameworks such as V2Ray have experimented with using QUIC as a transport layer for secure communication between nodes, as described in [V2R24] documentation. However, these solutions typically use QUIC only as a transport for proxy protocols rather than implementing a full IP-level VPN tunnel.

This protocol is also increasingly used in applications such as video streaming, online gaming, and real-time communication systems. These characteristics make QUIC particularly suitable for modern networking scenarios, which motivates the exploration of its use in VPN tunneling solutions.

3 Theoretical Fundamentals

3.1 Virtual Private Network

The term VPN “Virtual Private Network” refers to a technology that creates a secure and encrypted communication channel between a user's device and the Internet. By transmitting data through a protected virtual tunnel, a VPN protects information from unauthorized access and helps maintain user privacy and confidentiality. The use of a virtual private network ensures that only the person intended to receive the data will be able to access it since the service hides the IP address of the user. Because of the advantages offered, virtual private networks are widely utilized for corporate purposes. [Clo24]

[RFC27] describes VPNs as mechanisms that allow private communication to be established across existing IP infrastructures. Instead of using dedicated physical links, VPN technologies use shared public networks to transport traffic between remote endpoints while allowing the communicating devices to operate as if they were connected through a private network.

VPNs can be implemented in different ways depending on the communication requirements. The two most common approaches are Remote Access VPN and Site-to-Site VPN. A Remote Access VPN enables users to securely connect to a private network from a remote location through the Internet, as shown in Figure 1. Once the connection is established, users will be able to access the various resources within the private network as if they are connected to the local network. This type of VPN is commonly used by employees who work remotely. Users often rely on remote access VPNs to enhance their online privacy and access Internet services that may be restricted in certain regions. [Gee25]

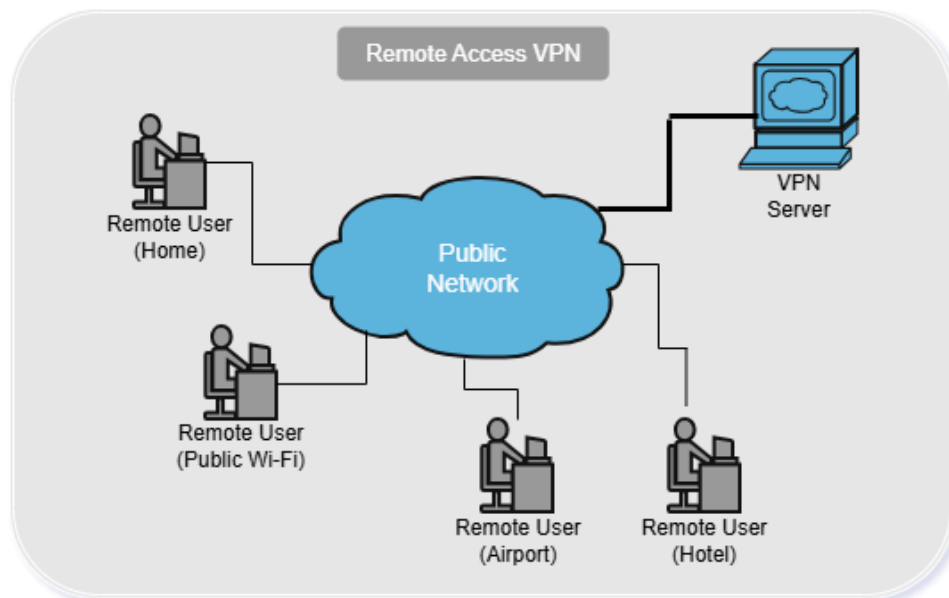


Figure 1. Remote Access VPN example [Kha23]

Unlike a Remote Access VPN, a Site-to-Site VPN is designed to securely interconnect multiple networks located in different geographical locations. Instead of connecting individual users, this approach creates a secure connection between different networks, allowing devices located at separate sites to communicate through the internet. Site-to-Site VPNs are frequently used by

organizations with multiple offices that need to exchange data securely [Com23]. Depending on the communication scenario, these VPNs can be used either to connect different branches of the same organization (intranet VPN) or to establish secure connectivity between separate organizations (extranet VPN). [Gee25]

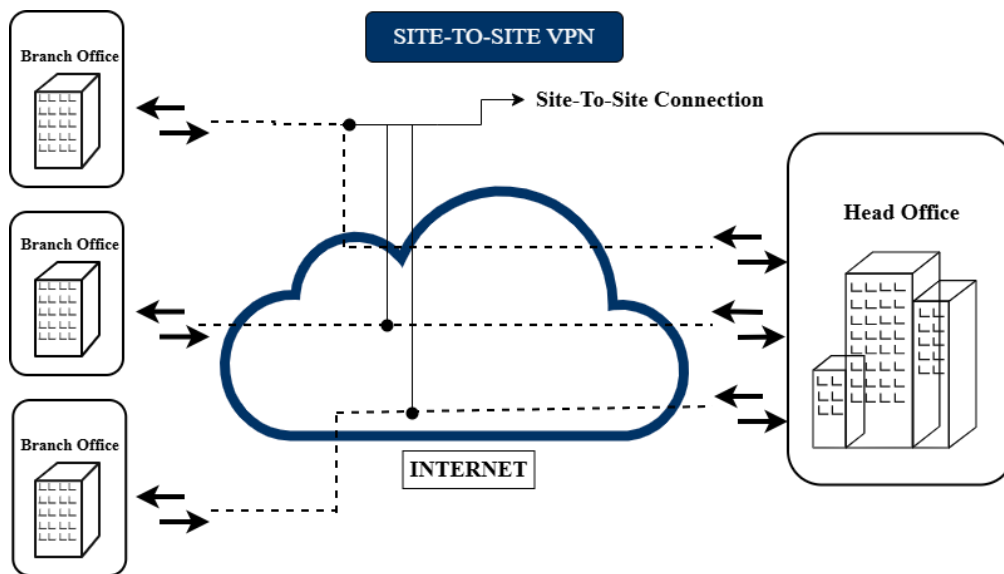


Figure 2. Site-to-Site VPN example [Com23]

The architecture of a Virtual Private Network (VPN) relies on several technologies that work together to ensure secure communication. One of the most important components of a VPN is the tunnel, which provides a safe path for data transmission across public networks. In addition to tunneling mechanisms, VPN architectures may also include components such as firewalls and proxy servers, depending on the requirements of the network. These elements contribute to traffic protection, access control, and communication management. [Ban25]

3.2 VPN tunneling

Network packets can be intercepted while being transmitted between endpoints which represents a challenge when exchanging information. To address this issue, VPN technologies rely on tunneling mechanisms that protect data during transmission. Instead of sending packets directly through the network, the traffic is first encrypted and then prepared for transport through a VPN tunnel.

The tunneling process involves communication between two endpoints. The two endpoints agree on the encryption parameters that will be used to protect communication before any data is exchanged. The original packet is then encrypted and encapsulated inside another packet before transmission.

Encapsulation can be described as the process of placing one network packet inside another packet. The original packet becomes the payload of a new packet that contains additional protocol information required for transport. In a typical VPN tunnel, an additional IP header is attached to

the packet. The outer header identifies the tunnel endpoints, while the inner header preserves the addressing information associated with the original communication.[Jyo23]

After encapsulation, the transmitted data is encrypted, transforming the original data into a coded form that can only be accessed by using the appropriate cryptographic keys. VPN solutions also rely on authentication mechanisms to verify the identity of the communicating parties before data exchange takes place. [VPN24]

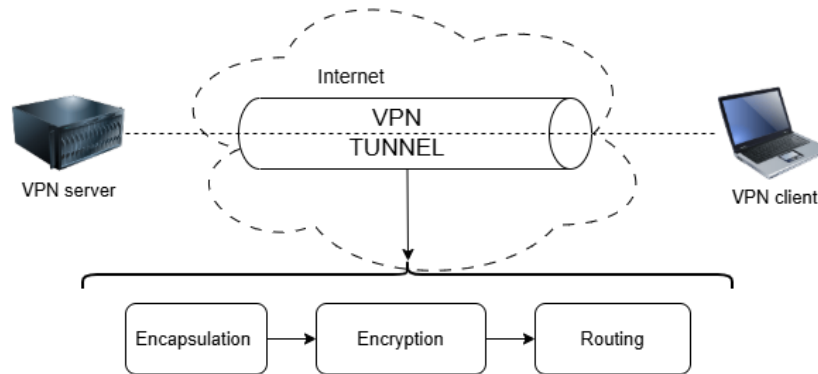


Figure 3. Main components of a VPN Tunnel

3.3 Virtual Network Interfaces

Virtual network interfaces are software-based interfaces that emulate the behavior of physical network interfaces within the operating system kernel [Gro25].

TUN interface is a virtual network interface that is commonly used in Linux systems and operates at the network layer (Layer 3), meaning that it can only be used for routing and can handle the transmission and reception of IP packets. Compared to the physical network interfaces which exchange traffic through dedicated hardware, the TUN device is created by the operating system kernel. Applications can read packets from the interface and write packets to it, allowing network traffic to be exchanged between user-space programs and the kernel networking stack. [Tas25]

The TUN interface is commonly used in VPN solutions because it allows IP traffic to be routed through a virtual tunnel. For example, VPN protocols such as OpenVPN use TUN interfaces to route traffic through virtual tunnels before it is transmitted across the network, creating virtual point-to-point links between remote endpoints. This allows VPN applications to intercept, process, and forward IP packets without requiring direct access to physical network hardware. TUN interfaces are also used in virtualization environments, where they help create isolated virtual networks for virtual machines and containers. In addition, they can also be utilized in network simulation and testing environments, allowing traffic to be captured, analyzed, and evaluated under different networking scenarios [Tas25]. If multiple local area networks must be bridged, TUN interfaces cannot be used because they operate at the network layer and process IP packets rather than Ethernet frames. Instead, TUN devices are primarily used to route traffic through virtual tunnels, which makes them suitable for VPN applications. TAP interfaces, on the other hand, operate at the data link layer and can be used to bridge different networks. [Wal21]

3.4 VPN Protocols – QUIC Protocol: Comparison with TCP and UDP, Architecture and Features

VPN connections can be established using several protocols, each relying on different mechanisms for transmission of data across the network. Some of the major VPN protocols include PPTP, OpenVPN, WireGuard, and IKEv2/IPsec. PPTP is one of the earliest VPN protocols and uses a TCP control channel together with a GRE tunnel to transport data [Palo]. OpenVPN is more flexible and can operate over either TCP or UDP, depending on the requirements of the network environment. WireGuard is a modern VPN protocol and relies only on UDP, which helps reduce protocol overhead and improve performance. IKEv2/IPsec also commonly uses UDP for communication and connection management.

Although these protocols differ in terms of architecture and functionality, many modern VPN solutions rely on TCP or UDP as their underlying transport mechanisms. For this reason, understanding the characteristics of these transport protocols is important before discussing the QUIC protocol.

Transport protocols enable communication between devices connected to a network. They are generally divided into two categories: connection-oriented and connectionless protocols. Connection-oriented protocols establish a connection between the communicating endpoints before data transmission and provide mechanisms that improve reliability. Connectionless protocols transmit data without creating a dedicated connection, focusing on simplicity and fast delivery. TCP (Transmission Control Protocol) and UDP (User Datagram Protocol) represent the two fundamental transport protocols used in network communications. TCP follows a connection-oriented approach while UDP follows a connectionless approach. [Kum20]

3.4.1 TCP protocol

Transmission Control Protocol (TCP) is a transport layer protocol designed to provide reliable communication between devices connected to a network. It is one of the core protocols of the Internet and is included in the standards developed by the Internet Engineering Task Force (IETF). Before data transmission begins, TCP establishes a connection between the endpoints and ensures that the exchanged information is delivered correctly and in the correct order. The protocol also includes mechanisms for error detection, flow control, and retransmission of lost packets, which makes it appropriate for applications where data reliability is needed. For this reason, TCP is widely used by many Internet services and application-layer protocols, including HTTP, FTP, SMTP, and SSH. [For25]

Despite its reliability when transmitting data and its usage in modern networks, TCP can introduce several limitations and delays. Some of the most important drawbacks are presented and described in the following section.

- Overhead and Latency

TCP provides reliable data delivery by dividing information into smaller segments and making sure that they are received in the correct order. If data is lost or corrupted during transmission, the

missing segments can be retransmitted. These mechanisms make TCP suitable for applications where reliability is more important than speed [Ade17]. However, the process of establishing and maintaining a connection introduces additional overhead, which may affect performance in scenarios involving frequent exchanges of small amounts of data [Kum20].

Before data transmission begins, TCP performs the three-way handshake to verify that both endpoints are ready to communicate. The client initiates the connection by sending a SYN segment to the server. The server responds with a SYN-ACK segment, acknowledging the request and indicating that it is ready to communicate. Finally, the client sends an ACK segment, completing the connection establishment procedure. The three-way handshake allows both endpoints to synchronize communication parameters and confirm that the connection is available before data transmission begins. [MDN25]

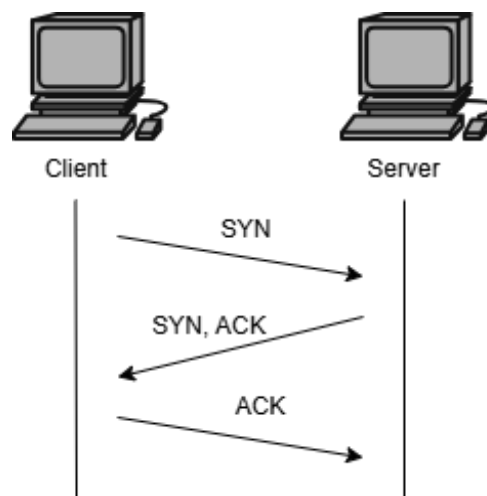


Figure 4. TCP Three-Way Handshake

Although this mechanism improves reliability, it also introduces additional latency. Connection establishment latency is commonly measured using the round-trip time (RTT), which represents the time required for a message and its response to travel between two endpoints. TCP connection establishment requires multiple RTTs before the connection is ready for communication, which increases the delay before data exchange can begin. [Kum20]

- Head-of-Line-Blocking

The Head-of-Line (HOL) blocking problem represents one of the limitations of TCP. Since TCP delivers data in order, packets that arrive successfully may still be delayed if an earlier packet is lost during transmission. In this situation, the missing packet must be retransmitted and received before the remaining packets can be processed. Figure 5 illustrates an example of HOL. Packet loss, network congestion, or packets taking different paths through the network can all contribute to this behavior. [Kum20]

Head-of-Line blocking can also affect application performance in real-world scenarios. For example, in HTTP/1.1, multiple resources of a webpage are often transferred through the same TCP connection. If the transmission of a large resource is delayed, smaller resources must wait

until the missing data is received and processed. As a result, webpage loading times may increase even when only a small portion of the transmitted data is affected.

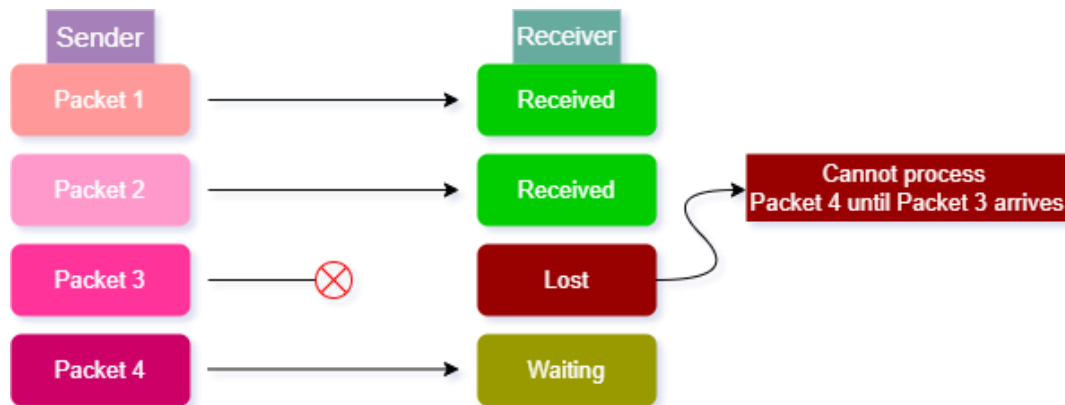


Figure 5. Head-of-Line Blocking

To reduce the impact of HOL blocking, browsers often establish multiple parallel TCP connections. While this approach can improve performance, it also increases resource consumption and may lead to additional network congestion. [Man18]

3.4.2 UDP Protocol

UDP is a transport protocol that does not require a connection to be established before data transmission. This reduces overhead and allows data to be exchanged with lower latency. Due to its lightweight design, UDP is commonly used in applications where fast delivery is more important than reliability, such as audio and video streaming. The protocol uses a smaller header and does not rely on handshake procedures, resulting in lower bandwidth consumption and reduced processing requirements. However, UDP does not guarantee packet delivery, packet ordering, or error recovery. As a result, reliability mechanisms must be implemented by higher-layer protocols when required. [Iya20]

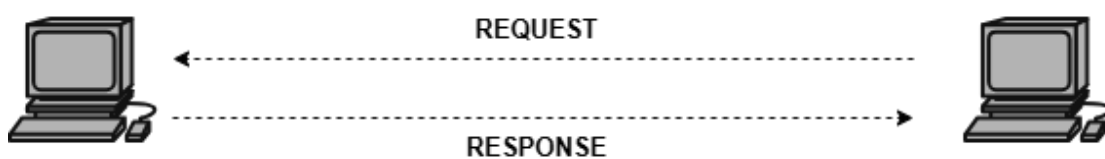


Figure 6. UDP connection establishment [Gee25]

3.4.3 QUIC protocol

QUIC is a modern transport protocol designed to provide secure, reliable, and low-latency communication. QUIC is implemented on top of UDP and integrates encryption directly into the protocol, unlike traditional transport protocols. It was originally developed to improve web performance and reduce connection establishment delays. One of the key features of QUIC is its ability to multiplex multiple data streams within a single connection. This helps reduce the impact of Head-of-Line (HOL) blocking and improves the efficiency of data transmission. In

addition, QUIC can be deployed without requiring modifications to the operating system, making it easier to adopt in modern network applications [Vie18].

Before discussing the main components and mechanisms of QUIC, it is useful to examine the structure of a QUIC packet. QUIC supports two header formats: a long header and a short header [Nay22]. Figure 7 illustrates an example of a QUIC packet using the long header format. The long header is mainly used during connection establishment, when the client and server exchange information required to create the connection and negotiate protocol parameters [Kum20]. Among the most important fields from the long header are the QUIC version, the Connection ID used to identify the connection, the packet length, and the packet number. Each packet is assigned a unique packet number, which plays an important role in loss recovery and packet tracking. The Connection ID allows a connection to remain active even when the underlying network changes, supporting features such as connection migration. [Nay22]

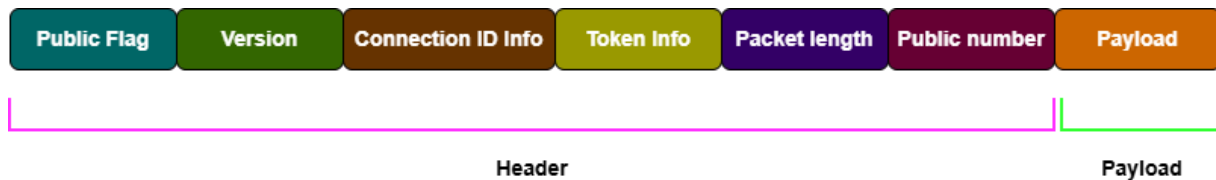


Figure 7. The structure of a QUIC packet with long header [Kum20]

Once the connection has been established and the required encryption keys have been negotiated, QUIC switches to the short header format. The short header is illustrated in Figure 8 and contains only the information required for normal data transmission, such as the Destination Connection ID and the packet number. By reducing the amount of information included in each packet, the short header helps decrease protocol overhead. [Tan24]

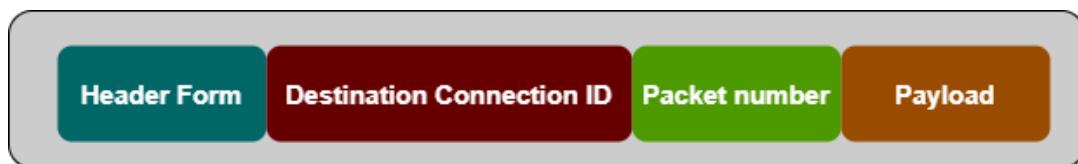


Figure 8. The structure of a QUIC packet with short header [Tan24]

The use of two different header formats allows QUIC to provide the additional information needed during connection while maintaining efficient packet processing during normal communication.

The payload section of a QUIC packet contains the information exchanged between the communicating endpoints. In short header packets, the payload is protected through encryption and contains QUIC frames. Different types of frames can be carried inside the payload, including STREAM frames, which transport application data and ACK frames, which are used to acknowledge received packets. During the connection establishment phase, QUIC packets can also carry information related to the TLS handshake, version negotiation, and retry mechanisms. Once the handshake is completed, most of the exchanged data is encrypted. Before the payload

can be interpreted, the information present in the header is used to correctly identify and process the packet.

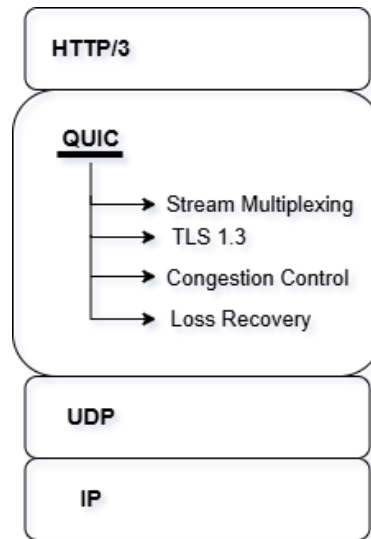


Figure 9. QUIC protocol components [Sal21]

Figure 9 illustrates the protocol stack used by QUIC. QUIC integrates several functionalities, including encryption, stream multiplexing, congestion control, and loss recovery. The following sections describe each of these components and their role within the protocol architecture.

- Stream Multiplexing

QUIC supports stream multiplexing which allows multiple independent data streams to be transmitted simultaneously over a single UDP connection. Unlike HTTP/2 over TCP, where packet loss in one stream can delay the delivery of data from other streams due to Head-of-Line (HOL) blocking, QUIC limits the impact of packet loss to the affected stream, allowing data from other streams to continue being delivered without interruption. [Vie18]

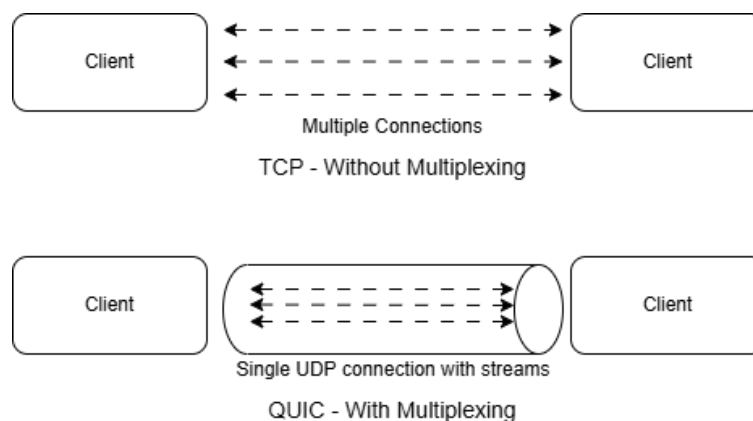


Figure 10. TCP vs UDP Connections [Kum20]

Each stream is handled as an independent logical flow within the same connection. QUIC maintains reliable and ordered delivery for the data belonging to a particular stream, while

keeping it separate from the others. This separation allows communication to continue on unaffected streams even when packet loss occurs on a different stream. [Chi20]

- TLS 1.3

Another important component of QUIC is its integration with TLS 1.3, which provides security for the communication process. Compared to previous TLS versions, TLS 1.3 reduces the time required to establish a secure connection. Under normal network conditions, a new connection can be established and secured within a single round-trip time (RTT). When a client reconnects to a server it has previously contacted, application data can be transmitted immediately using a zero round-trip time (0-RTT) setup. These improvements help reduce connection establishment latency while maintaining secure communication. [Tho21]

QUIC carries TLS handshake messages directly within the transport protocol. The two protocols work closely together: TLS is responsible for authentication and key generation, while QUIC provides reliable and ordered delivery of the exchanged information. This tight integration improves the interaction between the transport and security layers [Tho21]

QUIC uses an encrypted transport handshake to establish a secure connection between the client and the server. During the initial connection, the client and server exchange the information required for authentication and encryption. Compared to TCP/TLS, QUIC reduces the number of round trips needed to establish a secure connection. A typical QUIC connection uses a 1-RTT handshake, allowing application data to be exchanged sooner than in TCP/TLS, which generally requires three round trips before data transmission can begin. [Zha21]

As illustrated in Figure 11, when a client connects to a server for the first time, it sends a Client Hello (CHLO) message. The server replies with a rejection message (REJ) containing the information needed to continue the handshake. After validating this information, the client sends a complete CHLO message to the server. If the handshake is successful, the server responds with a Server Hello (SHLO) message and the connection is established. Once the handshake has been completed, encrypted data can be exchanged between the client and the server. [Zha21]

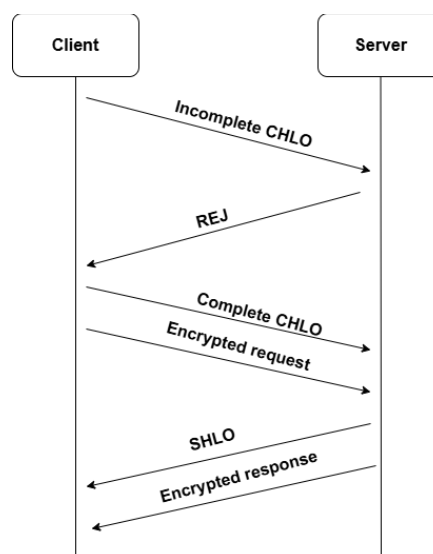


Figure 11. Initial 1-RTT handshake process QUIC [Zha21]

- Congestion Control

Congestion control is used to prevent network overload and maintain stable communication when network resources become limited. QUIC is not tied to a specific congestion control algorithm. Instead, it provides a flexible framework that allows different congestion control mechanisms to be implemented depending on the requirements of the application. QUIC also assigns a unique packet number to every transmitted packet. This helps detect packet loss more accurately and improves round-trip time (RTT) measurements, contributing to more efficient loss recovery and congestion management. [Nay22]

QUIC commonly uses congestion control algorithms derived from TCP, such as CUBIC, while still allowing alternative algorithms to be implemented. In addition, packet numbers increase monotonically throughout the lifetime of a connection and are never reused, including for retransmitted packets. [Kum20]

- Loss Recovery

Loss recovery mechanisms in QUIC, such as tail loss probes and fast retransmit, are designed to detect packet loss quickly and retransmit missing data without waiting for long timeout periods. To achieve this, QUIC uses more detailed acknowledgment information than TCP. Instead of acknowledging only the next expected byte, QUIC acknowledgments can describe which packets have been received and which are missing. They can support up to 256 NACK (Negative Acknowledgment) ranges which allows packet loss to be detected more accurately and reduces the time required to identify lost data. In addition, each transmitted packet is assigned a unique packet number, including retransmitted packets. This helps eliminate confusion during loss detection and improves the efficiency of packet recovery, particularly in network environments where packet loss occurs frequently. [CS24416]

3.4.4 Advantages of QUIC

Several of the advantages offered by QUIC have already been introduced through its main components. These include reduced Head-of-Line blocking, lower protocol overhead, and improved latency. As discussed earlier, the integration of TLS 1.3 is one of the key features of QUIC. Besides reducing connection establishment latency, it also enhances communication security by providing built-in encryption and authentication mechanisms. The use of unique session tokens for each connection helps prevent unauthorized access and improves protection against potential attacks. [Tix25]

Another significant advantage of QUIC is its ability to support connection migration. QUIC connections are identified using Connection IDs that are exchanged during the handshake process. This allows the connection to remain active even when the client's IP address changes. This feature is particularly useful for mobile devices. For example, as illustrated in Figure 12, when a user switches from a Wi-Fi network to a mobile data connection, the IP address assigned to the device may change. In traditional TCP-based communication, such a change can interrupt the connection and require it to be re-established, however, QUIC can continue using the existing connection without restarting the communication process. [Nay22]

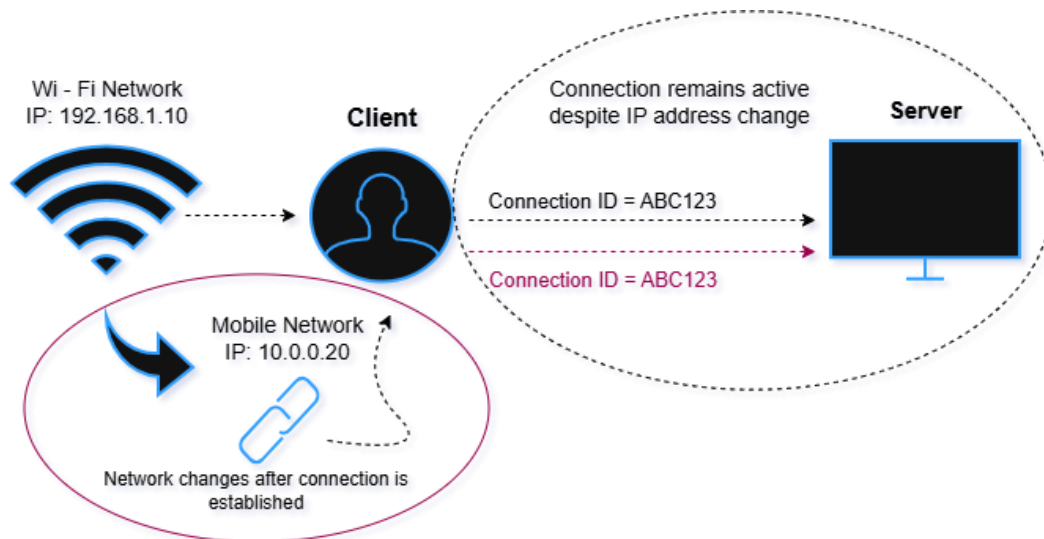


Figure 12. Example of QUIC connection migration

3.4.5 Comparison between TCP and QUIC protocol

The previous sections presented the main characteristics of both TCP and QUIC. Although both protocols provide reliable data transmission, they differ in aspects related to connection establishment, security, packet recovery, and communication efficiency. Table 1 summarizes the most important differences between TCP and QUIC.

Table 1. Comparison between TCP and QUIC

Features	TCP	QUIC
Transport Protocol	TCP	UDP-based
Connection Establishment	Three-way Handshake	1-RTT / 0-RTT handshake
Encryption	TLS implemented separately	TLS 1.3 integrated
Multiplexing	Not supported	Native stream multiplexing
Head-of-Line Blocking	Causes head-of-line blocking	Reduced through independent streams
Loss Recovery	Traditional acknowledge mechanism	Detailed ACK information and unique packet numbers
Connection Migration	Not supported	Supported through Connection IDs
Deployment	Usually requires OS-level implementation	Can be implemented in user space

As shown in Table 1, QUIC combines several advantages of both TCP and UDP. It provides reliability and security while reducing latency, supporting multiplexing and enabling connection migration. Based on the characteristics presented above, QUIC combines the low-overhead communication model of UDP with additional mechanisms for reliability, security, and connection management. These properties make QUIC a promising transport protocol for applications that require both performance and secure communication, including VPN tunneling solutions.

3.5 Network Address Translation (NAT)

Network Address Translation (NAT) is a networking technology that enables computers using private IP addresses to communicate with public networks. As the number of Internet users has increased, the limited availability of public IPv4 addresses has become a significant challenge. While enterprise and local networks typically use private IP addresses, communication over the Internet requires public addresses. NAT provides the translation mechanism that allows hosts within private networks to access external networks. In addition to addressing the shortage of public IPv4 addresses, NAT also prevents devices located on the public Internet from directly communicating with hosts that use private addresses, providing an additional level of isolation for internal networks. NAT is commonly implemented on routers connecting private networks to the Internet and can be configured using different translation methods depending on the network requirements.

IP addresses are divided into public and private address spaces. Public IP addresses are globally unique and are used for communication over the Internet. In contrast, private IP addresses are reserved for internal networks and cannot appear on the public Internet. Routers on the Internet do not maintain routes to private addresses. Three private address ranges are reserved for IPv4 networks: 10.0.0.0–10.255.255.255, 172.16.0.0–172.31.255.255, and 192.168.0.0–192.168.255.255. Organizations can select the appropriate private address range depending on the size of their internal network. Since private addresses are intended for isolated networks, different organizations may use the same address range provided that their networks do not need to communicate with one another.

The use of private addressing combined with NAT offers several advantages. By allowing multiple devices to share public addresses, NAT helps reduce the consumption of public IPv4 addresses. It also allows internal network addresses to remain unchanged if the Internet service provider is replaced, making Internet connectivity more flexible. Furthermore, because private addresses are not directly reachable from the public Internet, internal networks are less exposed to external access. Despite these advantages, NAT also has several limitations. Since address translation requires modifying packet information and maintaining translation records, it introduces additional processing compared to standard packet forwarding and consumes router resources. Another limitation is that replacing private addresses with public addresses prevents end-to-end IP tracking, as external systems only observe the translated public address. In addition, computers located on the public Internet cannot directly initiate communication with hosts inside private networks unless port mapping is configured. Some applications are also incompatible with NAT because they do not allow intermediate devices to modify packets.

Several types of NAT can be used depending on the required functionality. Static NAT establishes a fixed one-to-one mapping between a private and a public IP address, allowing communication in both directions but without reducing the number of required public addresses. Dynamic NAT uses a pool of public addresses and temporarily assigns an available address to a private host whenever external communication is required. Once the communication ends, the public address is returned to the pool for reuse. Network Address Port Translation (NAPT) further improves address utilization by translating both IP addresses and port numbers, allowing multiple private hosts to share a single public IP address. To achieve this, the router maintains a table that records the address and port translations used during communication. [Hua22]

4 Implementation

4.1 Brief Presentation

The proposed architecture of the developed VPN solution is presented in Figure 13. The main objective of this project was to implement a secure communication system capable of transporting IP packets through a tunnel based on the QUIC protocol. To achieve this objective, technologies such as QUIC, Linux TUN virtual interfaces and Python were employed in the development of the solution.

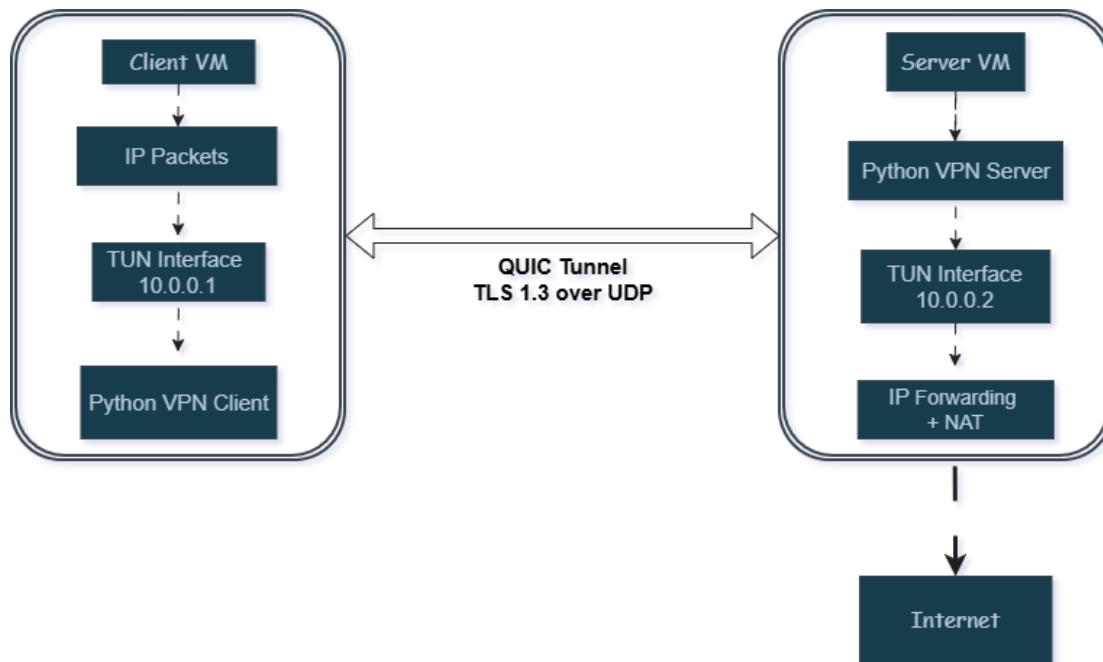


Figure 13. Architecture of the implemented QUIC VPN solution

The proposed solution consists of two main entities, a client and a server, connected through a secure QUIC tunnel. The implementation was developed and tested in a virtualized environment using Oracle VM VirtualBox. Two Ubuntu virtual machines were deployed, one acting as the VPN client and the other as the VPN server. The client is responsible for capturing IP traffic generated by local applications and forwarding it through a secure QUIC connection, while the server receives the traffic and routes it towards the destination network.

The operation of the system begins with the initialization of the server, which listens for incoming QUIC connections. Once the server is running, the client establishes a connection and a secure communication channel is created between the two endpoints. After the connection is established, IP packets generated on the client side are captured through a virtual TUN interface, encapsulated into QUIC streams and transmitted to the server. The server extracts the packets and injects them into its own TUN interface, allowing them to be processed by the operating system networking stack. In order to enable communication with external networks, IP forwarding and Network Address Translation (NAT) are configured on the server. As a result, packets received through the VPN tunnel can be routed to the Internet and their source addresses translated before leaving the server. Traffic returning from the network follows the same path in the opposite direction, ensuring bidirectional communication through the VPN tunnel.

4.2 Preparing the local environment

The implementation of the proposed VPN solution was carried out in a virtualized environment. For the deployment of the virtual environment, Oracle VM VirtualBox was installed from the official website <https://www.virtualbox.org/wiki/Downloads>. VirtualBox is an open-source virtualization platform that allows multiple operating systems to run simultaneously on a single physical machine. After the installation of VirtualBox, to create the development environment, the Ubuntu 24.04.4 LTS operating system image was downloaded from the official Ubuntu website <https://ubuntu.com/download/desktop> and installed on a virtual machine.

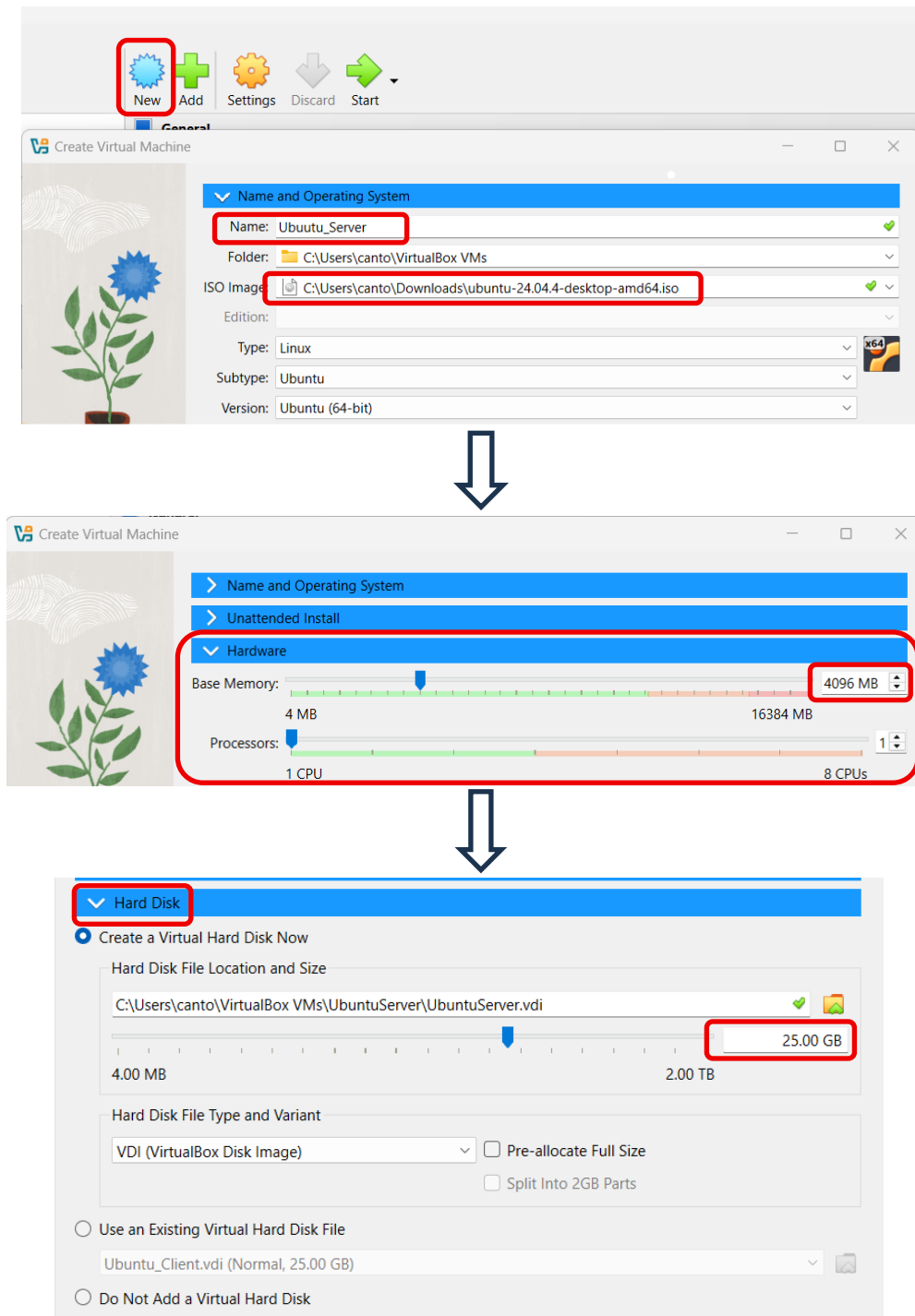


Figure 14. Main steps for creating the virtual machine in Oracle VM VirtualBox.

Figure 14 illustrates the process of creating the virtual machine used during the implementation of the VPN solution. In order to create a new virtual machine, the New button in Oracle VM VirtualBox was selected. Next, the Ubuntu 24.04.4 LTS ISO image was chosen as the installation source, the whole process being described later in this section, and the virtual machine was assigned the name *Ubuntu Server*. At this stage, the virtual machine itself was only being configured, while the Ubuntu operating system had not yet been installed. During the hardware configuration stage, 4096 MB of RAM and one virtual processor were allocated to the system. In the next step, a virtual hard disk with a capacity of 25 GB was created to store the operating system and all project-related files.

The next step consists of attaching the Ubuntu installation image to the virtual machine. As shown in Figure 16, this is done from the Storage settings by selecting the empty optical drive, choosing Choose a Disk File, and attaching the Ubuntu 24.04.4 LTS ISO image. After this step, the virtual machine is ready for the Ubuntu installation.

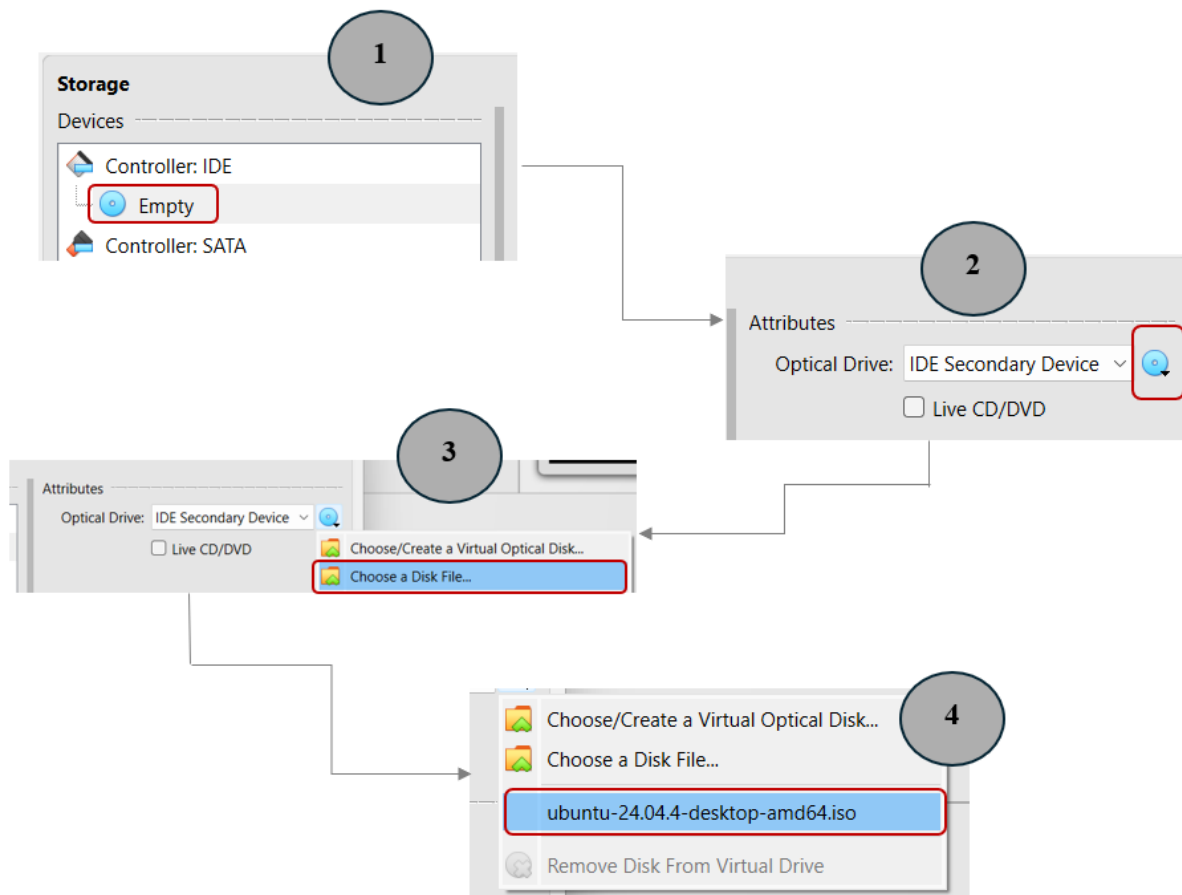


Figure 15. Steps for attaching the Ubuntu ISO image in Oracle VM VirtualBox

After the virtual machine had been configured, it was started and the Ubuntu installation process began. The operating system was installed on the previously created virtual hard disk, and a user account was configured during the installation. Once the installation was completed, Ubuntu was ready to be used. A clone of this virtual machine was then created, resulting in two identical virtual machines that were used as the client and the server throughout the implementation. As shown in Figure 16, the primary network adapter of each virtual machine was configured in Bridged Adapter mode, configuration which allows both the client and server virtual machines to appear as independent hosts on the local network and communicate directly with each other. The use of a Bridged Adapter was required in order to establish the QUIC connection between the two endpoints and to perform the subsequent VPN functionality tests.

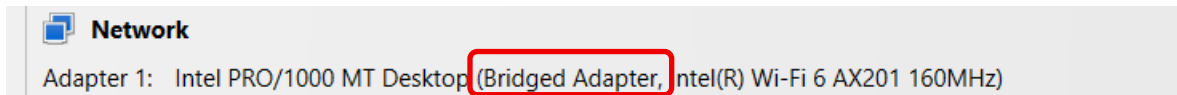


Figure 16. Virtual Machine configured to a Bridged Adapter

4.3 Installing and Configuring the Required Software Components for the VPN Solution

After the virtual machines were successfully created and configured, the next step consisted of installing and configuring the software components required for the implementation of the VPN. Python 3 and the associated package management tools were installed on both the client and server virtual machines. The commands used for the installation and verification of Python 3 are presented below:

```
sudo apt update
sudo apt install python3
sudo apt install python3-pip
```

Figure 17. Commands used for installing Python3

```
python3 --version
Python 3.12.3
```



Verification of the installed Python 3
version

Figure 18. Commands used for the verification if installation

The aioquic library was installed for the implementation of the QUIC protocol and secure communication between the client and server components. The implementation of the VPN solution relies on the aioquic Python library and it includes support for the QUIC protocol, TLS 1.3 encryption and HTTP/3. In the proposed implementation, aioquic was used to establish encrypted QUIC connections, manage bidirectional data streams and transport IP packets between the client and server through the VPN tunnel. The commands used for the installation and verification of the aioquic library are presented below:

```
pip install aioquic
```

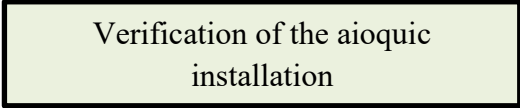


aioquic installation

Figure 19. Commands used for installing the aioquic library

```
pip3 show aioquic
Name: aioquic
Version: 1.3.0
Summary: An implementation of QUIC and HTTP/3
Home-page:
Author:
Author-email: Jeremy Lainé <jeremy.laine@m4x.org>
License:
Location: /home/denisa/.local/lib/python3.12/site-packages
Requires: certifi, cryptography, pyrsnpack, pyopenssl, service-identity
Required-by:
```

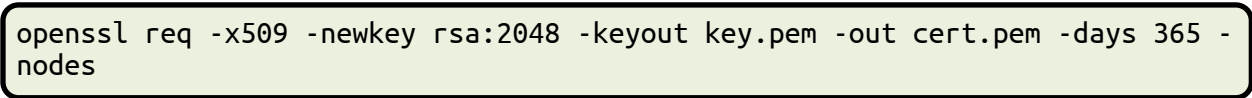




Verification of the aioquic
installation

Figure 20. Verification of the aioquic library installation

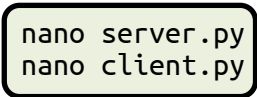
Furthermore, a digital certificate and its corresponding private key were generated on the server virtual machine to support the secure communication mechanisms provided by the QUIC protocol. During the connection establishment phase, these cryptographic credentials are used to authenticate the server and enable the TLS 1.3 handshake process. Once the handshake is completed, all data exchanged between the client and server is transmitted through an encrypted communication channel. The command used to generate the certificate and private key is presented below:



```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes
```

Figure 21. Commands used for generating the certificate and private key

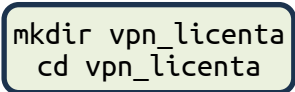
After generating the cryptographic certificate and private key on the server virtual machine, the Python source files used for the VPN implementation were created. The Nano text editor was used to create and edit the application source code directly from the terminal. The following command was executed on both the client and server virtual machines in order to create the corresponding Python files:



```
nano server.py  
nano client.py
```

Figure 22. Commands used for creating
Python files

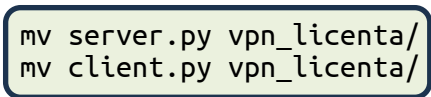
To keep the project files organized, a directory named `vpn_licenta` was created on both virtual machines. This was done using the Linux `mkdir` command, followed by the `cd` command to navigate to the newly created directory. The commands used for this step are shown below.



```
mkdir vpn_licenta  
cd vpn_licenta
```

Figure 23. Commands used for creating a directory

Once the nano files had been created, they were moved into the `vpn_licenta` directory using the `mv` command, keeping all project-related files in a single location. The corresponding command is shown below.



```
mv server.py vpn_licenta/  
mv client.py vpn_licenta/
```

Figure 24. Command used for
moving files

4.4 Initial QUIC Communication Setup

Before implementing the VPN functionality, a simple QUIC client-server application was developed in order to understand the connection establishment process provided by the QUIC protocol. The purpose of this preliminary implementation was to verify the correct operation of the aioquic library and to observe the QUIC handshake procedure between two virtual machines.

The server application was configured to listen for incoming QUIC connections on port 4433 using a digital certificate and a private key generated previously. On the client side, a connection request was initiated towards the server. Once the handshake process was successfully completed, a QUIC connection was established between the two endpoints. At this stage, no user data was exchanged between the client and the server. The implementation was limited to connection establishment and verification of the secure communication channel, providing the foundation for the subsequent development of the VPN functionality. The source code of both components is presented below.

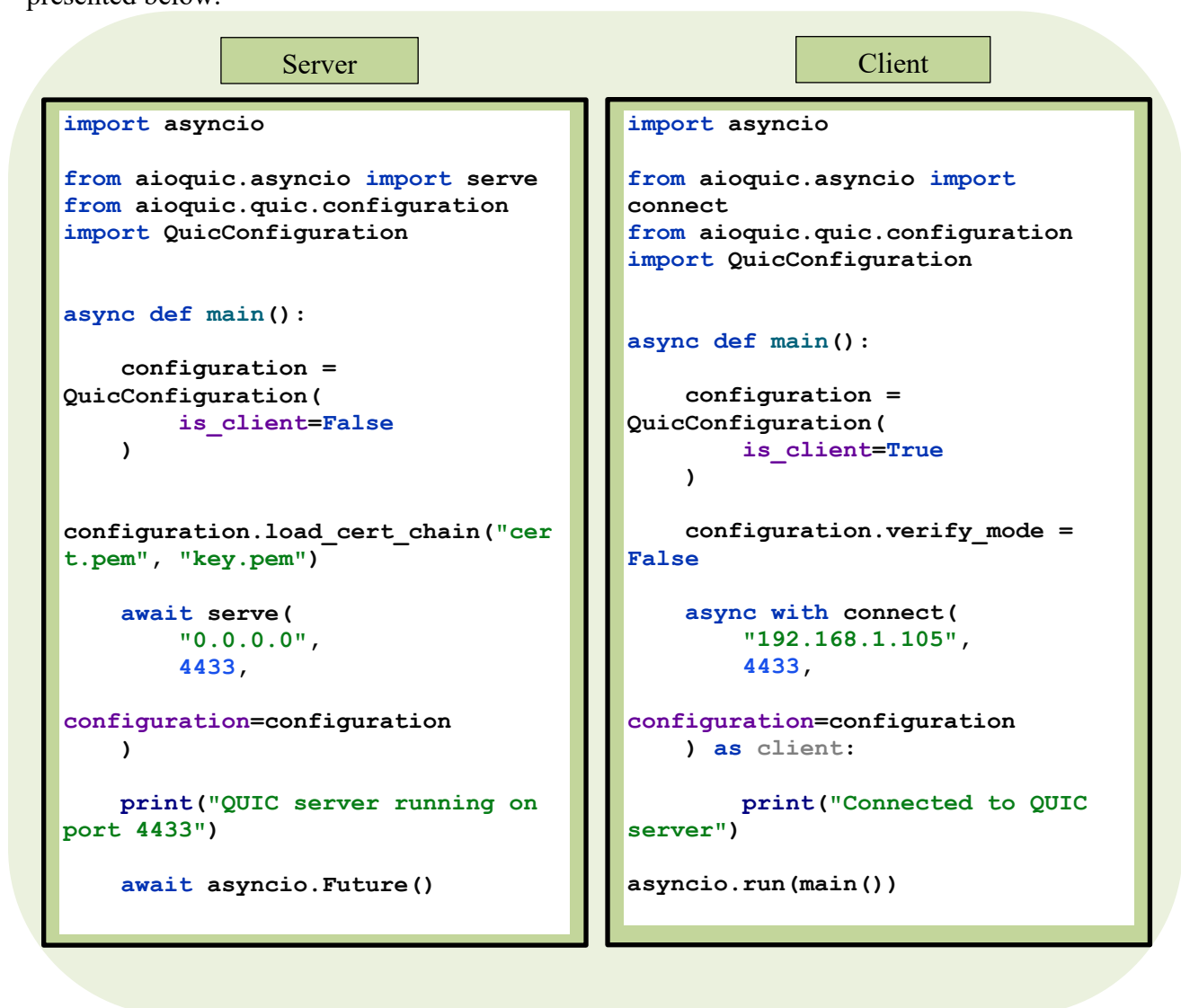


Figure 25. Initial Server and Client Codes

The server application creates a QUIC endpoint capable of accepting incoming connections. A `QuicConfiguration` object is initialized with the `is_client` parameter set to `False`, indicating that the application operates as a server. The previously generated certificate and private key are

loaded using the `load_cert_chain()` function, enabling the server to participate in the TLS 1.3 handshake. Finally, the `serve()` function binds the application to port 4433 and waits for incoming connection requests.

On the client side, a separate `QuicConfiguration` object is created with the `is_client` parameter set to `True`. Since the server uses a self-signed certificate generated specifically for testing purposes, certificate verification is disabled on the client by setting `verify_mode` to `False`. This allows the client to accept the server certificate without requiring it to be signed by a trusted Certificate Authority (CA). The `connect()` function is then used to establish a connection with the server using its IP address and listening port. Once the QUIC handshake is successfully completed, an encrypted communication channel is established between the two virtual machines, and the message "Connected to QUIC server" is displayed to confirm that the connection has been successfully created. At this stage, the implementation is limited to connection establishment, while the transmission of IP packets is introduced in the following development steps.

Figure 26 illustrates the initial stage of the implementation. This stage represents the starting point for the development of the VPN solution, before integrating the TUN interfaces and packet forwarding functionality.

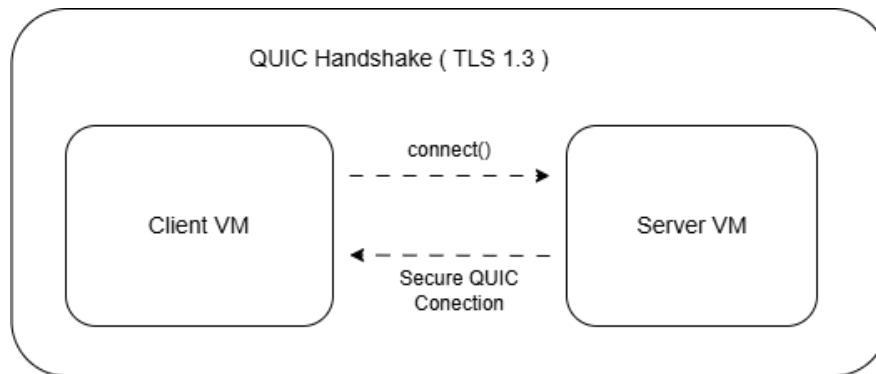


Figure 26. Initial implementation of the QUIC client-server connection

4.5 Integration of Virtual TUN Interfaces

The basic QUIC client-server implementation described in the previous section provided only the functionality required to establish a secure connection between the two virtual machines. In order to transform this connection into a VPN solution capable of transporting IP packets, virtual TUN interfaces were integrated into both the client and server applications.

TUN interfaces operate at the network layer and allow user-space applications to exchange raw IP packets with the operating system. By using a pair of TUN interfaces, the VPN application is able to capture IP packets generated by the client, transmit them through the QUIC tunnel and inject them back into the network stack on the server side. Below is listed the server code part.

```
def create_tun() :

    TUNSETIFF = 0x400454ca
    IFF_TUN = 0x0001
    IFF_NO_PI = 0x1000

    tun = os.open("/dev/net/tun", os.O_RDWR)
```

```

ifr = struct.pack('16sH', b'tun0', IFF_TUN | IFF_NO_PI)

fcntl.ioctl(tun, TUNSETIFF, ifr)

flags = fcntl.fcntl(tun, fcntl.F_GETFL)
fcntl.fcntl(tun, fcntl.F_SETFL, flags | os.O_NONBLOCK)

return tun

```

The `create_tun()` function is responsible for creating and configuring a virtual TUN interface on the server virtual machine. The `/dev/net/tun` device is opened and configured using the `TUNSETIFF` `ioctl` operation. The `TUNSETIFF` `ioctl` operation instructs the Linux kernel to create a virtual network interface named `tun0` and configure it as a TUN device. This interface acts as the entry and exit point for IP packets transported through the VPN tunnel. The `IFF_TUN` flag specifies that a TUN interface should be created, while `IFF_NO_PI` disables the additional packet information header, allowing only raw IP packets to be exchanged. Finally, the interface is configured in non-blocking mode in order to avoid blocking the execution of the application while waiting for packets.

After the virtual TUN interface is created, several initialization steps are performed in order to prepare it for VPN operation, as shown in the code below.

```

self.running = True
self.tun = create_tun()
self.buffer = b""

subprocess.run(
    ["ip", "addr", "add", "10.0.0.2/24", "dev", "tun0"],
    stderr=subprocess.DEVNULL
)
subprocess.run(
    ["ip", "link", "set", "tun0", "up"]
)

```

First, the variable `self.running` is set to `True`, indicating that the VPN session is active and that packet processing should continue as long as the connection remains established. The file descriptor returned by the `create_tun()` function is then stored in `self.tun`, allowing the application to subsequently read and write IP packets through the virtual interface. In addition, an empty buffer is initialized and assigned to `self.buffer`. This buffer will later be used to temporarily store data received through QUIC streams before complete IP packets are reconstructed and processed.

Once the interface has been successfully created, it must be assigned an IP address. This is achieved through the Linux command `ip addr add 10.0.0.2/24 dev tun0`, which is executed directly from the Python application using the `subprocess.run()` function. The address `10.0.0.2` represents the server endpoint of the VPN tunnel, while the `/24` subnet mask defines the virtual network used for communication between the client and the server. The parameter `stderr=subprocess.DEVNULL` is used to suppress any error messages generated by the command, preventing unnecessary output from being displayed in the terminal. After assigning the IP address, the virtual interface is activated using the command `ip link set tun0 up`. By default, a newly created network interface is in the DOWN state and cannot transmit or receive packets. Executing this command changes the interface state to UP, making it operational and ready to participate in network communication. At this stage, the server-side TUN interface is fully configured and prepared to exchange IP packets through the VPN tunnel.

The client implementation uses a similar function, named `open_existing_tun()`, to access and configure the virtual TUN interface. Since the same Linux TUN/TAP driver and configuration parameters are employed, the operation is identical to the one previously described for the server-side implementation. The function returns a file descriptor that allows the client application to exchange raw IP packets with the operating system through the `tun0` interface.

```
def open_existing_tun():
    TUNSETIFF = 0x400454ca
    IFF_TUN = 0x0001
    IFF_NO_PI = 0x1000

    tun = os.open("/dev/net/tun", os.O_RDWR)
    ifr = struct.pack('16sh', b'tun0', IFF_TUN | IFF_NO_PI)
    fcntl.ioctl(tun, TUNSETIFF, ifr)

    flags = fcntl.fcntl(tun, fcntl.F_GETFL)
    fcntl.fcntl(tun, fcntl.F_SETFL, flags | os.O_NONBLOCK)

    return tun
```

Similar to the server implementation, the client application opens and configures the `tun0` virtual interface using the `open_existing_tun()` function. An IP address (10.0.0.1/24) is then assigned to the interface and the interface is activated using the `ip link set tun0 up` command. Finally, the command `ip route add default dev tun0` configures the TUN interface as the default route, ensuring that all outgoing client traffic is redirected through the VPN tunnel. This allows IP packets to be captured by the application and securely transmitted to the server through the QUIC connection. The code segment that performs these operations on the client side is presented below.

```
tun = open_existing_tun()
print("TUN created on client")
subprocess.run(
    ["ip", "addr", "add", "10.0.0.1/24", "dev", "tun0"],
    stderr=subprocess.DEVNULL
)

subprocess.run(
    ["ip", "link", "set", "tun0", "up"]
)

subprocess.run(
    ["ip", "route", "add", "default", "dev", "tun0"],
    stderr=subprocess.DEVNULL
)
```

Figure 27 represents an illustration of the implementation after the integration of the virtual TUN interfaces on both the client and the server. Unlike the previous stage, where only a secure QUIC connection was established, the current implementation creates a virtual VPN tunnel that enables the transmission of IP packets through the encrypted QUIC connection.

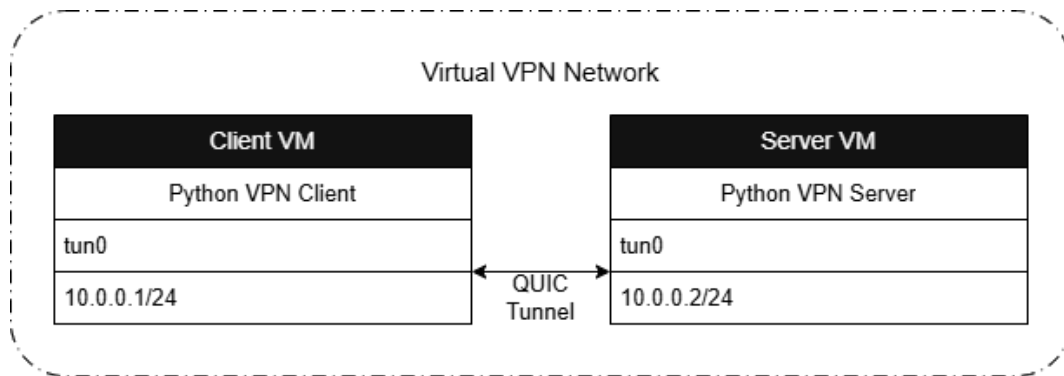


Figure 27. Configuration of the virtual TUN interfaces used by the VPN implementation

4.6 Packet Transmission Through QUIC Streams on the Client Side

Once the client and server TUN interfaces have been configured, the next step consists of transmitting the captured IP packets through the QUIC connection. This functionality is implemented by the `tun_to_quic()` function, which continuously reads packets from the client-side TUN interface, encapsulates them into QUIC stream data and forwards them to the remote endpoint. In this way, network traffic generated by the operating system can be transported through the encrypted QUIC tunnel established between the two virtual machines. The implementation of this functionality is presented below and it was implemented in the client script.

```

async def tun_to_quic(client, tun):

    stream_id = client._quic.get_next_available_stream_id()

    while True:
        try:
            packet = os.read(tun, 1500)

        except BlockingIOError:
            await asyncio.sleep(0.01)
            continue

        if not packet:
            continue

        data = struct.pack("!H", len(packet)) + packet

        client._quic.send_stream_data(
            stream_id,
            data,
            end_stream=False
        )

    client.transmit()

```

The function begins by obtaining a new QUIC stream identifier through the `get_next_available_stream_id()` method. QUIC supports the use of multiple independent streams within a single connection, allowing data to be exchanged efficiently while avoiding unnecessary blocking between different communication flows. In the proposed implementation, a dedicated stream is created and used for transmitting the IP packets captured from the client-side TUN interface.

A continuous loop is then executed in order to permanently monitor the virtual interface for outgoing traffic. The instruction `os.read(tun, 1500)` retrieves packets directly from the TUN interface, where 1500 bytes correspond to the Maximum Transmission Unit (MTU) commonly used in Ethernet networks. If a different MTU is configured for the TUN interfaces the buffer size should be adjusted accordingly so that complete IP packets can still be read and forwarded correctly. Each packet read from the interface represents a complete IP packet generated by the operating system. Since the TUN interface was previously configured in non-blocking mode, situations may occur in which no packets are immediately available. In such cases, a `BlockingIOError` exception occurs and the application temporarily suspends execution using `asyncio.sleep(0.01)` before attempting to read again. This approach prevents unnecessary CPU utilization while ensuring continuous monitoring of the interface.

Once a valid packet is available, it is prepared for transmission through the QUIC tunnel. Before sending the packet, its length is calculated and encoded into a two-byte field using the `struct.pack("!H", len(packet))` function. This length field is appended to the beginning of the packet, allowing the receiving endpoint to accurately determine the boundaries of each packet and reconstruct the original data after reception, even if multiple packets are received consecutively within the same QUIC stream.

The resulting data is transmitted using the `send_stream_data()` method. The parameter `end_stream=False` indicates that the QUIC stream remains active after each transmission, allowing additional packets to be sent through the same stream without establishing a new one. Once the data has been placed in the QUIC transmission buffer, the `transmit()` function instructs the QUIC stack to immediately send the buffered information to the remote endpoint. Through this mechanism, every IP packet captured by the client-side TUN interface is encapsulated into the encrypted QUIC connection and securely delivered to the VPN server.

4.7 Reception and Reconstruction of QUIC Packets on the Server Side

After the client transmits packets through the QUIC stream, the server must receive, reconstruct and inject them into its local TUN interface. This functionality is implemented within the `quic_event_received()` method, which processes incoming QUIC events generated by the aioquic library. Whenever stream data is received, the server extracts the encapsulated IP packets and forwards them to the operating system through the virtual network interface. The corresponding implementation is presented below.

```
if isinstance(event, StreamDataReceived):

    self.buffer += event.data

    while len(self.buffer) >= 2:

        pkt_len = struct.unpack("H", self.buffer[:2])[0]

        if len(self.buffer) < 2 + pkt_len:
            break

        packet = self.buffer[2:2 + pkt_len]
        self.buffer = self.buffer[2 + pkt_len:]

        if len(packet) < 20 or packet[0] >> 4 != 4:
            continue

        os.write(self.tun, packet)
```

The processing begins when a *StreamDataReceived* event is generated by the QUIC library, indicating that data has been received from the client. The received bytes are appended to the buffer stored in *self.buffer*, allowing the application to accumulate data even when packets arrive fragmented across multiple QUIC frames.

Before a packet can be reconstructed, the code verifies that at least two bytes are available in the buffer. These two bytes represent the packet length field that was added by the client before transmission. Using the *struct.unpack()* function, the server extracts this value and determines the size of the packet that follows. If the complete packet has not yet been received, processing is temporarily suspended until additional data arrives.

Once a complete packet is available, it is extracted from the buffer and removed from the stored data. An additional validation step is then performed to ensure that the received information corresponds to a valid IPv4 packet. This is achieved by verifying both the minimum IPv4 header size and the version field contained in the packet header.

After successful validation, the packet is written directly to the server-side TUN interface using the *os.write()* function. At this point, the operating system processes the packet as if it had been received from a physical network interface. Consequently, packets transmitted through the QUIC tunnel become available to the server networking stack and can be forwarded towards their destination.

4.8 IP Forwarding and NAT Configuration

Once the packets are received from the QUIC tunnel and injected into the server-side TUN interface, they become available to the Linux networking stack. However, in order for these packets to reach external networks, additional routing functionality must be enabled. For this reason, IP forwarding and Network Address Translation (NAT) were configured on the VPN server. These mechanisms allow packets received through the VPN tunnel to be forwarded to the Internet and enable returning traffic to be correctly delivered back to the client. The implementation of this configuration is presented below.

```
subprocess.run(
    ["sysctl", "-w", "net.ipv4.ip_forward=1"],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL
)

subprocess.run(
    ["iptables", "-t", "nat", "-A", "POSTROUTING",
     "-o", "enp0s3", "-j", "MASQUERADE"],
    stderr=subprocess.DEVNULL
)
```

The first command enables IPv4 forwarding by setting the *net.ipv4.ip_forward* kernel parameter to 1. By default, Linux systems do not forward packets between network interfaces and therefore behave as end hosts rather than routers. Enabling IP forwarding allows the server to route packets received through the VPN tunnel towards external networks.

The second command configures Network Address Translation using the Linux iptables framework. More specifically, a rule is added to the POSTROUTING chain of the NAT table, applying the MASQUERADE action to packets leaving through the physical network interface

enp0s3. This operation replaces the source IP address of outgoing packets with the IP address of the server, allowing them to be routed correctly through the external network.

Together, IP forwarding and NAT transform the VPN server into a gateway between the virtual VPN network and the Internet. Packets received from the client can therefore be forwarded to external destinations, while response packets are automatically translated and routed back through the VPN tunnel to the originating client.

4.9 Complete VPN Operation and Packet Flow

The complete operation of the proposed VPN solution can be described as a sequence of consecutive steps that enable the secure transmission of IP packets between the client and the server through the QUIC protocol.

The communication process begins when the QUIC server is started. The server loads its TLS certificate and private key, then starts listening on port 4433 for incoming client connections. Afterwards, the client is started. It opens the tun0 interface, prepares the QUIC configuration, disables certificate verification because a self-generated certificate is used during development, and initiates a secure connection to the server.

After the QUIC handshake is successfully completed, both endpoints configure their TUN interfaces. The client assigns the IP address 10.0.0.1/24, while the server assigns 10.0.0.2/24. The server also enables IP forwarding and configures NAT, allowing packets received through the VPN tunnel to be forwarded to the external network. On the client, the default route is changed so that all outgoing traffic is redirected to the TUN interface.

To verify the implementation, a ping command is executed from the client towards 10.0.0.2. Since the default route points to the TUN interface, the operating system writes the generated ICMP packet into tun0 instead of sending it through the physical network interface. The client reads the packet from the TUN interface, adds a two-byte field containing the packet length, encapsulates it into a QUIC stream, encrypts it, and transmits it over the underlying UDP connection. When the data reaches the server, it is automatically decrypted, and the original IP packet is reconstructed using the transmitted length field. The packet is then injected into the server's TUN interface, where it is processed by the Linux networking stack. If necessary, the packet is forwarded to its destination through the physical network interface according to the routing table and NAT configuration.

The response follows the same path in the opposite direction. The server reads the packet from its TUN interface, prefixes it with its length, encapsulates it into the existing QUIC stream, encrypts it, and sends it back to the client. After receiving the data, the client decrypts it, reconstructs the original IP packet, and injects it into its own TUN interface. Finally, the operating system delivers the packet to the ping application, confirming that the packet has successfully traveled through the encrypted VPN tunnel between the client and the server. When the QUIC connection is terminated, both applications stop packet forwarding, close their TUN interfaces and exit.

5 Experimental Results

5.1 Experimental Demonstration of the VPN Implementation

This chapter presents the experimental evaluation of the implemented VPN application. The objective is to show how the client-server architecture works by analysing each stage of the communication process.

The experimental evaluation begins by launching the VPN server application. Once executed, the server initializes the QUIC endpoint and starts listening for incoming connections on port 4433, as indicated by the message *"QUIC server running on port 4433"*, shown in the first step from Figure 20. After the server has been successfully initialized, the client application is launched. The client immediately establishes a QUIC connection with the server, resulting in the successful creation of the encrypted communication channel. At the same time, the virtual TUN interfaces are configured on both endpoints, as indicated by the messages *"TUN created on client"* and *"TUN created on server"*. Following the successful connection, the terminal outputs *"Sent to server: 48"* on the client side and *"Sending packet to client: 48"* on the server side indicate that the applications have started processing and forwarding packets through the VPN infrastructure. The value 48 represents the size, expressed in bytes, of the transmitted IP packet. These packets correspond to background network traffic generated by the operating system and confirm that packet capture and forwarding mechanisms are active even before the connectivity tests are performed. The outputs are presented in Figure 28.

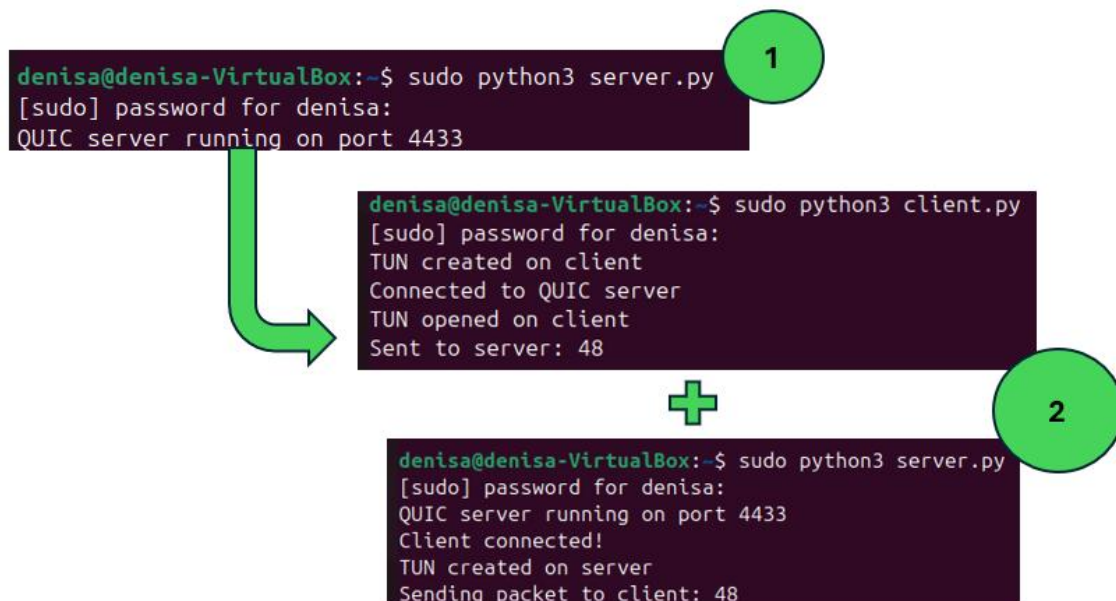


Figure 28. Terminal outputs illustrating the initialization of the QUIC VPN application

Before generating any user traffic, the tcpdump command was executed on the tun0 interface of both the client and the server in order to monitor the traffic passing through the virtual network interfaces. As illustrated bellow in Figure 29 and as mentioned before, the interface immediately begins capturing a small amount of background network traffic generated by the operating system. The captured packets correspond mainly to IPv6 Router Solicitation messages automatically generated by the Linux operating system when the virtual network interface becomes active. The presence of these packets confirms that the virtual TUN interface has been successfully created and is operational. However, these packets are not generated by the

implemented VPN application and therefore do not represent the transmission of user data through the tunnel.

Server

```
denisa@denisa-VirtualBox:~$ sudo tcpdump -i tun0
[sudo] password for denisa:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
17:03:02.730915 IP6 denisa-VirtualBox > ip6-allrouters: ICMP6, router solicitation, length 8
17:03:34.475207 IP6 denisa-VirtualBox > ip6-allrouters: ICMP6, router solicitation, length 8
```

Client

```
denisa@denisa-VirtualBox:~$ sudo tcpdump -i tun0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
17:02:39.623306 IP6 denisa-VirtualBox > ip6-allrouters: ICMP6, router solicitation, length 8
17:02:48.327085 IP6 denisa-VirtualBox > ip6-allrouters: ICMP6, router solicitation, length 8
```

Figure 29. Background traffic captured on the server and client TUN interface before generating user traffic

After confirming that both the client and the server have been successfully initialized, the connectivity of the VPN tunnel was verified using the ping command on another terminal. The client sends ICMP Echo Request packets to the server virtual address (**10.0.0.2**), while the server responds with ICMP Echo Reply packets. Since the default route of the client is configured through the tun0 interface, all ICMP packets are captured by the VPN application, encapsulated into the encrypted QUIC tunnel, transmitted to the server and injected into the server-side TUN interface. The corresponding terminal outputs are presented in Figure 30.

```
denisa@denisa-VirtualBox:~$ ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=8.75 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=13.4 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=10.6 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=8.63 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=14.6 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=9.08 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=10.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=10.9 ms
```

1

```
denisa@denisa-VirtualBox:~$ sudo python3 server.py
QUIC server running on port 4433
Client connected!
TUN created on server
Sending packet to client: 48
Sending packet to client: 48
Injected into TUN: 84
Sending packet to client: 84
Injected into TUN: 84
```

2

```
denisa@denisa-VirtualBox:~$ sudo python3 client.py
TUN created on client
Connected to QUIC server
TUN opened on client
Sent to server: 48
Sent to server: 84
Received from server: 84
Sent to server: 84
Received from server: 84
Sent to server: 48
Sent to server: 84
Received from server: 84
```

3

Figure 30. ICMP packet transmission through the QUIC VPN tunnel and the corresponding terminal outputs

From Figure 30 the first picture shows the successful execution of the ping command from the client towards the server virtual address (10.0.0.2). Every transmitted ICMP Echo Request receives a corresponding Echo Reply, confirming that the virtual VPN tunnel provides end-to-end connectivity between the two virtual machines. The measured round-trip times, ranging approximately between 8 ms and 20 ms, indicate that packets are transmitted through the encrypted QUIC tunnel with a low communication delay. The picture labeled with 2 from the figure presents the client-side debug output generated during the ping test. The messages *"Sent to server: 84"* indicate that ICMP packets captured from the tun0 interface are encapsulated and transmitted through the QUIC connection. The corresponding *"Received from server: 84"* messages confirm that the ICMP Echo Reply packets have been received from the server, extracted from the QUIC stream and delivered back to the client TUN interface. The value of 84 bytes corresponds to the complete IPv4 packet size transmitted through the VPN tunnel. Each message *"Injected into TUN: 84"*, as shown in the third picture, represents the server-side processing of the received packets and confirms that the packet extracted from the QUIC stream has been written into the server-side TUN interface, allowing the Linux networking stack to process it normally. The *"Sending packet to client: 84"* messages correspond to the ICMP Echo Reply packets generated by the server and transmitted back to the client through the encrypted QUIC tunnel.

To further verify that the VPN tunnel operates correctly, the tcpdump command was executed on the tun0 interface of both the client and the server while the ping command was running. This allows the packets travelling through the virtual tunnel to be observed directly. The captured traffic is presented in Figure 31.

```
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
17:48:06.966276 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 223, length 64
17:48:06.978215 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 223, length 64
17:48:07.967144 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 224, length 64
17:48:07.982530 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 224, length 64
17:48:08.969631 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 225, length 64
17:48:08.985104 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 225, length 64
17:48:09.970985 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 226, length 64
17:48:09.981698 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 226, length 64
17:48:10.971576 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 227, length 64
17:48:10.991034 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 227, length 64
17:48:11.973808 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 228, length 64
17:48:11.989812 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 228, length 64
17:48:12.974732 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 229, length 64
17:48:12.987751 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 229, length 64
17:48:13.976558 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 230, length 64
17:48:13.992599 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 230, length 64
17:48:14.978376 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 231, length 64
17:48:14.982482 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 231, length 64
17:48:15.980166 IP denisa-VirtualBox > 10.0.0.2: ICMP echo request, id 4725, seq 232, length 64
17:48:15.987252 IP 10.0.0.2 > denisa-VirtualBox: ICMP echo reply, id 4725, seq 232, length 64
```

Figure 31. ICMP packets captured on the client-side TUN interface.

Figure 31 shows the packets captured on the client-side tun0 interface. The request and reply share the same identifier (id) and sequence number (seq), making it possible to match every response to the request that generated it. As the test continues, the sequence number increases (223, 224, 225, ...), showing that packets are transmitted continuously and in the correct order. On the server side, the same communication can be seen from the opposite direction, where the requests are received and the replies are transmitted back to the client. The identifier and sequence number remain consistent throughout the communication, while the sequence number increases for each transmitted packet. Each captured packet also includes information such as the source and destination addresses and the packet length, which in this experiment is 64 bytes.

After stopping the ping command, a summary of the communication session is displayed, as shown in Figure 32. In this test, *451 packets* were transmitted and *451 packets* were received, resulting in *0% packet loss*. This confirms that every packet successfully travelled through the VPN tunnel and reached its destination. The summary also includes the *round-trip time (RTT)* statistics. The average RTT was 14.050 ms, while the minimum and maximum values were *2.123ms* and *36.141ms*. These results show that the implemented VPN provides stable communication with a relatively low delay throughout the experiment.

```
^C
--- 10.0.0.2 ping statistics ---
451 packets transmitted, 451 received, 0% packet loss, time 450690ms
rtt min/avg/max/mdev = 2.123/14.050/36.141/4.954 ms
```

Figure 32. Ping statistics obtained through the QUIC VPN tunnel

When the client disconnects, the server detects that the QUIC connection has been closed and displays the message *"Client DISCONNECTED"*. However, the server does not terminate. Instead, it remains active, displays *"QUIC server running on port 4433"*, and waits for a new client connection, as shown in Figure 33.

```
Sending packet to client: 84
Injected into TUN: 84
Sending packet to client: 84
Client DISCONNECTED
QUIC server running on port 4433
```

Figure 33. Server output after the client disconnects

5.2 Comparison of QUIC and TCP Traffic

This section presents a comparison between the network traffic generated by the implemented QUIC-based VPN and a simple TCP client-server application. Both applications were executed under the same network conditions, while Wireshark was used to capture the exchanged packets. The purpose of this comparison is to observe how the two protocols establish a connection and how their packet structure differs during communication.

5.2.1 QUIC Traffic

The traffic generated by the implemented QUIC-based VPN is presented first. Figure 34 shows the first packet exchanged between the client and the server after the connection is initiated. This packet is of type Initial and contains the information required to establish the QUIC communication.

No.	Time	Source	Destination	Protocol	Length	Info
101	72.025175385	192.168.1.109	192.168.1.105	QUIC	1242	Initial, DCID=d39d5c7465718fc1, SCID=7f964698227ae3eb, PKN: 0, CRYPTO
102	72.225127190	192.168.1.109	192.168.1.105	QUIC	1242	Initial, DCID=d39d5c7465718fc1, SCID=7f964698227ae3eb, PKN: 1, CRYPTO
103	72.436592377	192.168.1.105	192.168.1.109	QUIC	1242	Handshake, DCID=7f964698227ae3eb, SCID=8ce942b963312846
104	72.436902012	192.168.1.105	192.168.1.109	QUIC	431	Handshake, DCID=7f964698227ae3eb, SCID=8ce942b963312846

Figure 34. First packet exchanged between the client and the server

The packet uses the Long Header format, which is specific to the first packets exchanged in a QUIC connection. It contains the QUIC version and in this implementation Version 1 is used. Another important part of the header is represented by the Destination Connection ID and the Source Connection ID, both having a length of 8 bytes. The Packet Number field is also included in the header. Each QUIC packet receives a unique number, allowing the protocol to keep track of transmitted packets and detect losses if they occur. The header is followed by the payload section, which carries the information exchanged between the client and the server. In the Initial packet, the payload does not contain application data. Instead, it transports the information required during the connection establishment phase, including the CRYPTO frame used for the TLS 1.3 handshake. Since these messages are protected by encryption, Wireshark cannot display their contents and only indicates the encrypted payload. All this information is represented in Figure 35.

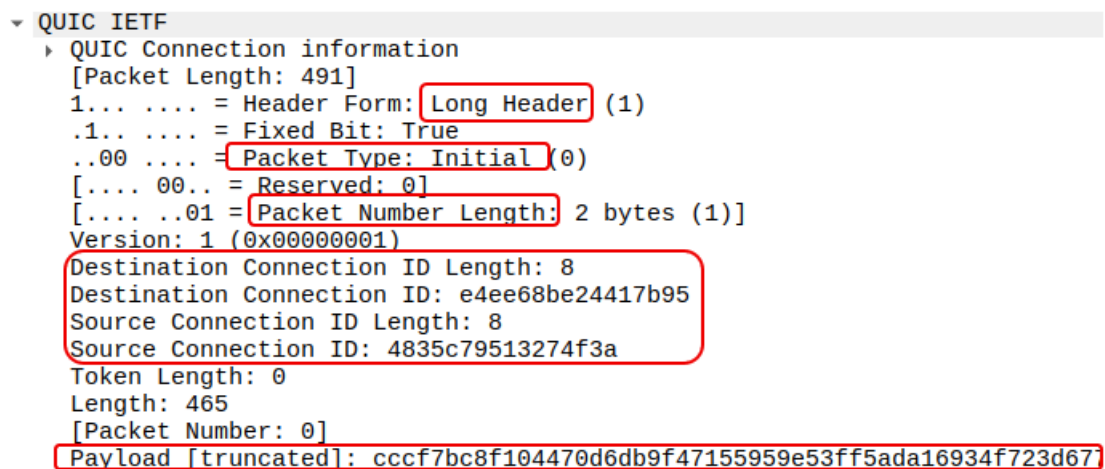


Figure 35. Header and payload fields of the first QUIC Initial packet.

This packet also contains a CRYPTO frame, as mentioned before. This frame transports the cryptographic information exchanged during the handshake. In this example, the CRYPTO frame carries the TLS 1.3 Client Hello message, which is the first message transmitted by the client. It contains the TLS version supported by the client, the available cipher suites, supported cryptographic groups, signature algorithms and the key exchange parameters required to negotiate the secure communication session. Figure 36 illustrates the CRYPTO frame.



Figure 36. CRYPTO frame carrying the TLS 1.3 Client Hello message.

Figure 37 presents another QUIC packet captured after the connection has already been established. Unlike the Initial packet described previously, this packet uses the QUIC Short Header format, indicating that the handshake has been completed and the communication is in the data transmission phase. As shown in the capture, the UDP source port is 4433, while the destination port is 53642, indicating that the packet is sent from the server to the client. The packet has a total length of 41 bytes, with a QUIC packet length of 33 bytes. Only the Destination Connection ID is included in the header, reducing the protocol overhead compared to the Long Header used during connection establishment. The remaining bytes represent the encrypted payload, which contains the transmitted application data. Since the payload is encrypted, Wireshark cannot decode its contents and displays it only as encrypted data.

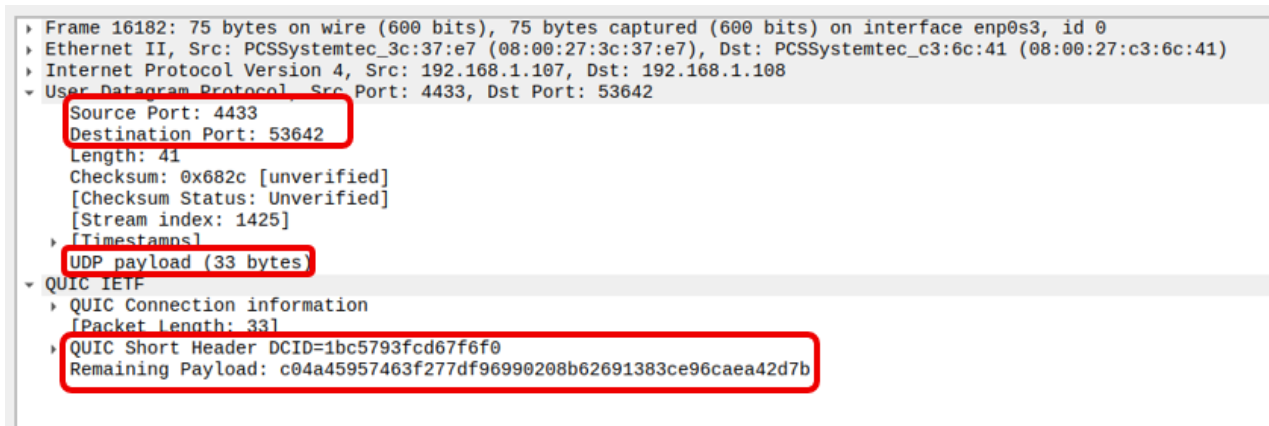


Figure 37. QUIC Short Header packet captured after connection establishment.

5.2.2 TCP Traffic

To analyse the TCP communication, the tcp filter was applied in Wireshark. This displays only the packets exchanged using the TCP protocol, making it easier to observe how the connection is established and how data is transmitted between the client and the server. Figure 38 shows the first TCP packet captured during the communication.

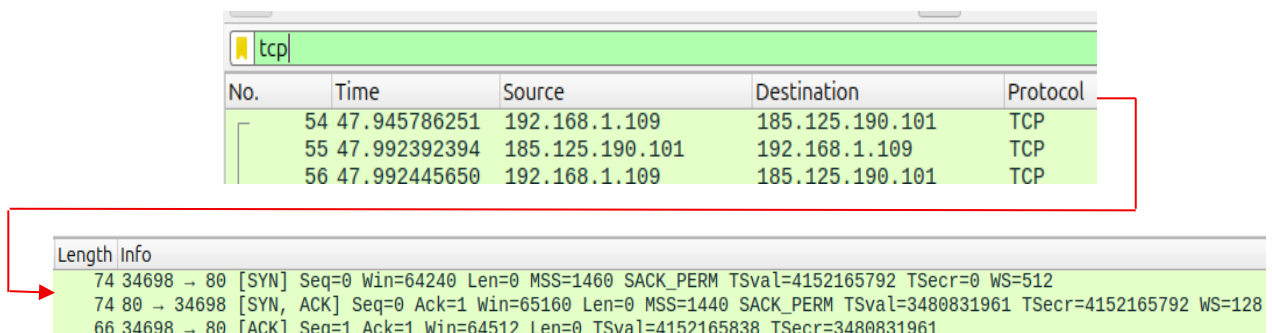


Figure 38. First TCP SYN packet captured during connection establishment

The first packet is a SYN packet sent by the client to initiate the TCP connection. It contains the source port (41388) and the destination port (4433), which identify the two communicating endpoints. The destination port corresponds to the port on which the server is listening, while the source port is automatically assigned by the operating system.

The Sequence Number is set to 0 because this is the first packet sent by the client. It represents the initial sequence number used by TCP to keep track of transmitted data. At this point, the

Acknowledgment Number is also equal to 0 because the client has not yet received any packets from the server. The most important field in this packet is the Flags section. In this example, only the SYN flag is set, indicating that the client is requesting the establishment of a new TCP connection. No application data is carried by this packet, as its only purpose is to begin the communication. The Window field specifies the amount of data that the receiver is currently able to accept without sending an acknowledgment. This value is used by TCP to perform flow control and prevent the receiver from becoming overloaded. The packet contains several TCP options, including the Maximum Segment Size (MSS), Selective Acknowledgment (SACK), Timestamp and Window Scale. These parameters are exchanged when the connection is established and are used to optimize the communication between the client and the server. Figure 39 illustrates the main fields of the TCP packet.

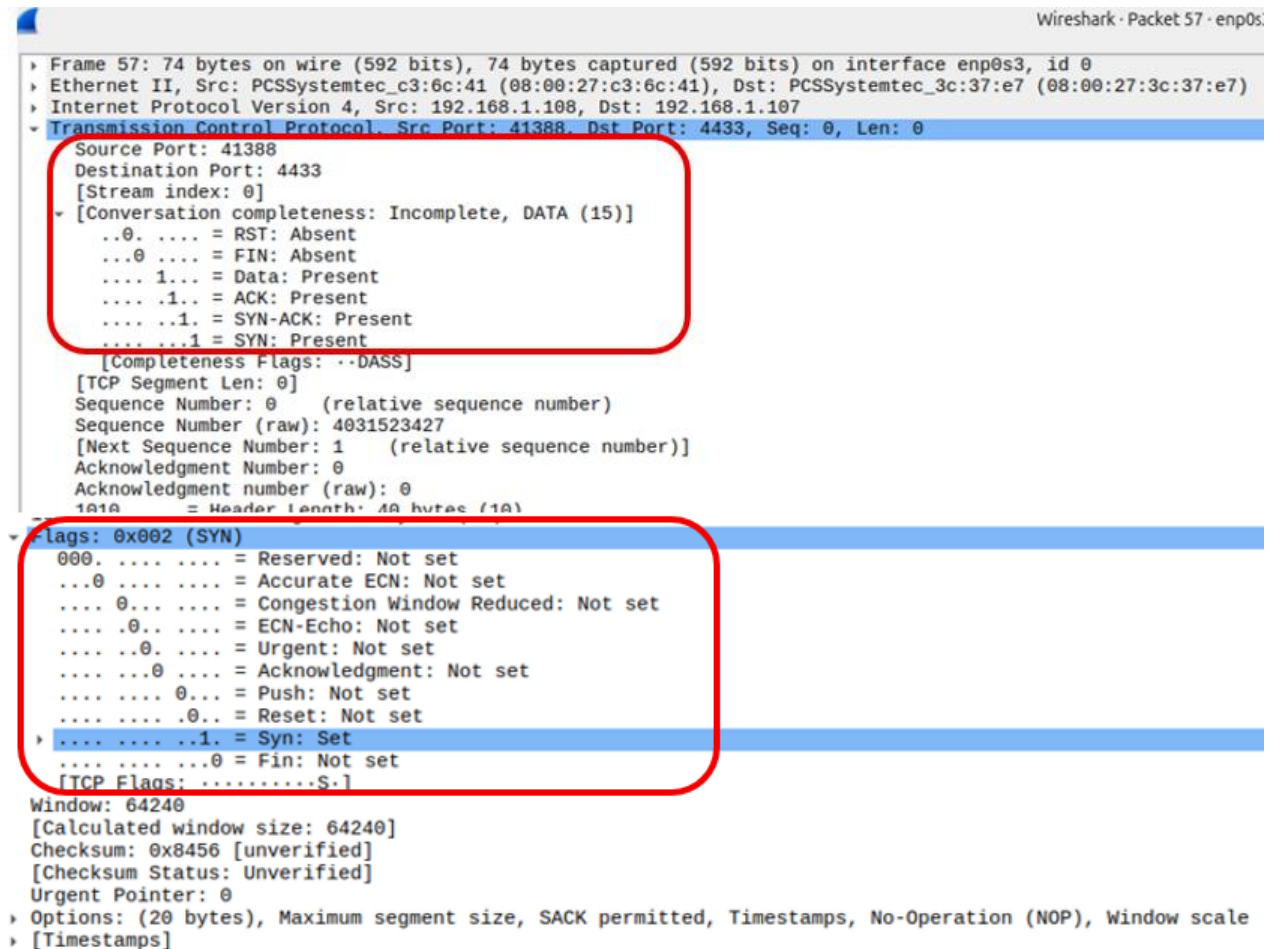


Figure 39. Main fields of the TCP SYN packet

5.2.3 Differences between QUIC and TCP

A major difference between QUIC and TCP can already be observed in the first packet exchanged between the client and the server. While QUIC begins the communication using a single Initial packet, TCP first performs a three-way handshake before any application data can be transmitted. Table 2 summarizes the main differences observed between the QUIC and TCP traffic captured during the experiments.

Table 2. Quic and TCP traffic comparison

Observed feature	QUIC traffic	TCP traffic
------------------	--------------	-------------

Transport protocol	UDP packets	TCP packets
First packet	Initial packet	SYN packet
Header format	Long Header during connection establishment, followed by Short Header after the handshake	Same TCP header format throughout the communication
Connection establishment	Begins with an Initial packet that also carries the TLS handshake	Begins with the TCP three-way handshake (SYN, SYN-ACK, ACK)
Connection identifier	Source Connection ID and Destination Connection ID	Source IP, destination IP, source port and destination port
Packet numbering	Packet Number	Sequence Number and Acknowledgment Number
Handshake information	TLS 1.3 Client Hello is carried inside the QUIC Initial packet	The first packets contain only TCP handshake information
Payload	Encrypted payload carried inside QUIC packets	Application data transmitted after the three-way handshake
Packet overhead	Long Header only during setup; Short Header during data transmission	Same header format used throughout the communication

Overall, the captured traffic shows that QUIC and TCP follow different approaches when establishing a connection and exchanging data. QUIC integrates security directly into the protocol and reduces header overhead after the connection has been established, while TCP relies on a separate three-way handshake and maintains the same header structure throughout the communication.

5.3 Comparison with Existing VPN Solutions

Connection establishment latency represents the time required to establish a communication session between the client and the server. The latency of the implemented QUIC VPN was measured in Wireshark over multiple runs, and the obtained results are presented in Figure 40.

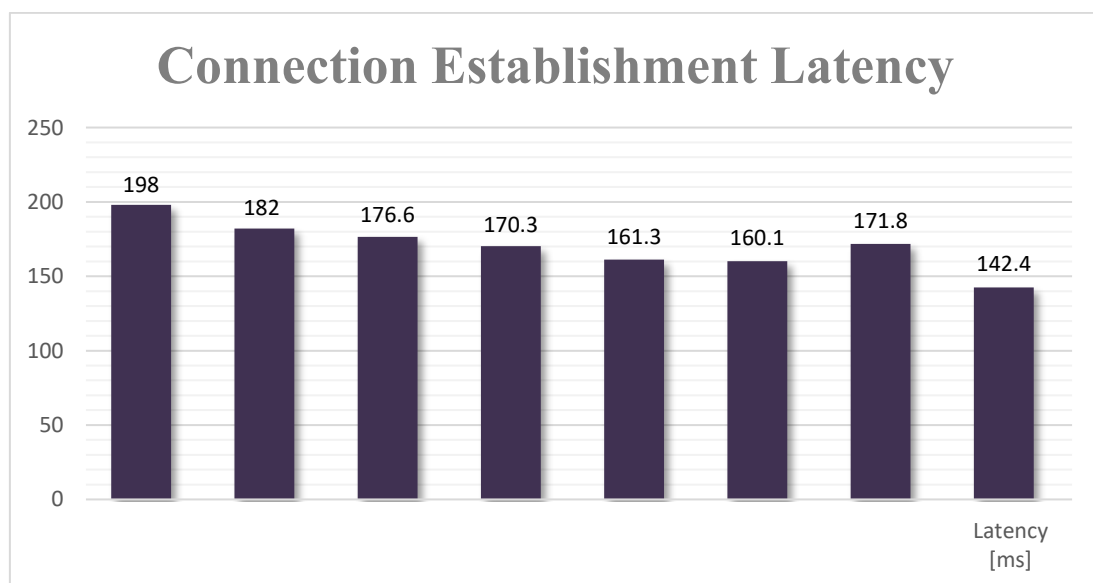


Figure 40. Latency measured over multiple runs

For each experiment, the connection establishment time was measured as the interval between the first QUIC Initial packet and the first protected application packet (1-RTT). To better position the implemented VPN solution, it is compared with one widely used VPN technology: OpenVPN. OpenVPN was also installed and configured on the same virtual machines and the connection establishment latency was measured for the VPN solution using the same experimental setup and measurement procedure. The average connection establishment latency across the eight measurements was 170.3 ms and it was used in this comparison. Figure 41 presents the results obtained for the two VPN solutions.

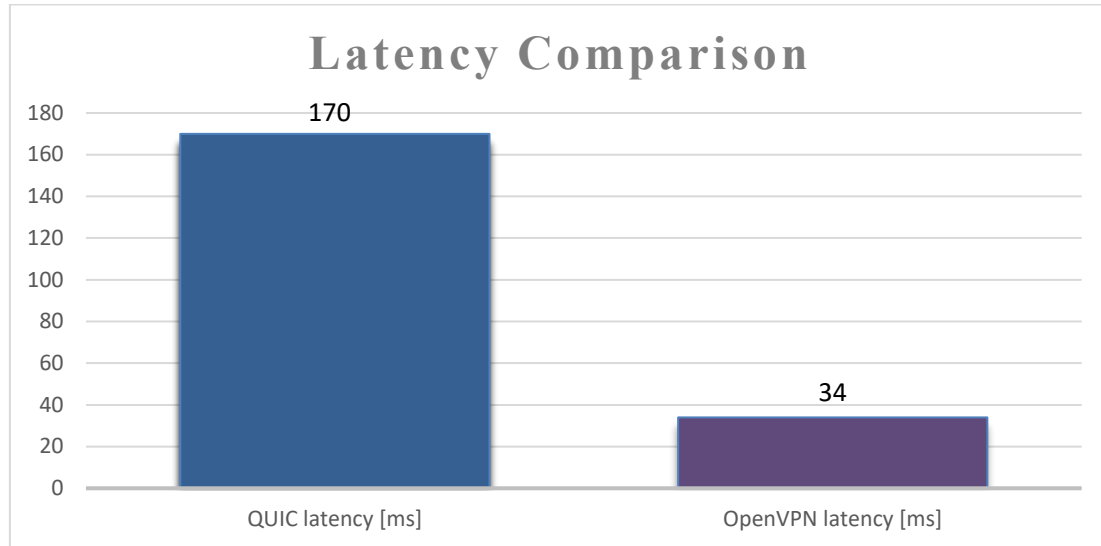


Figure 41. Latency Comparison between QUIC VPN and OpenVPN

The average connection establishment latency measured for the experimental QUIC VPN implementation was approximately 170 ms, while OpenVPN required around 34 ms to complete the connection establishment procedure. Although the proposed implementation produces a higher latency, this result is expected considering that it was developed as an experimental implementation in Python using the aioquic library, without the extensive optimizations found in mature VPN software. OpenVPN has been actively developed and optimized for many years and includes highly efficient implementations of cryptographic operations, packet processing, buffering mechanisms and connection management. The proposed QUIC VPN focuses primarily on demonstrating the feasibility of transporting IP packets through a QUIC tunnel rather than achieving maximum performance.

A comparison between the architecture of the proposed QUIC VPN implementation and two VPN technologies, WireGuard and OpenVPN is also presented below. The comparison highlights the main architectural characteristics of each solution, including the underlying transport protocol, the type of virtual interface used, and the tunnel implementation.

Table 3. Differences between the implemented VPN solution and the widely used VPN technologies (WireGuard and OpenVPN)

Feature	Implemented QUIC VPN	WireGuard	OpenVPN
Transport protocol	QUIC over UDP	UDP	UDP or TCP
Virtual interface	TUN	TUN	TUN or TAP
Tunnel type	Encrypted IP tunnel	Encrypted IP tunnel	Encrypted IP tunnel

Connection establishment	QUIC handshake with integrated TLS 1.3	NoiseIK cryptographic handshake	TLS handshake
Encryption	TLS 1.3 integrated into QUIC	ChaCha20-Poly1305	TLS with configurable cipher suites
Packet forwarding	Supported through the TUN interface	Supported	Supported
NAT support	Implemented using IP forwarding and NAT	Supported	Supported
Client-server architecture	Yes	Yes	Yes
Connection migration	Supported by the QUIC protocol through Connection IDs	Not natively supported	Not natively supported
Implementation purpose	Educational implementation demonstrating QUIC-based VPN communication	Production-ready VPN solution	Production-ready VPN solution

As shown in Table 3, the implemented VPN shares several characteristics with WireGuard and OpenVPN, including the use of a virtual TUN interface and encrypted IP tunnels. The main difference is the transport protocol: the proposed implementation uses QUIC over UDP with an integrated TLS 1.3 handshake, WireGuard uses UDP together with the NoiseIK protocol, while OpenVPN supports both UDP and TCP with a TLS-based handshake. Although WireGuard and OpenVPN are mature VPN solutions intended for production environments, the proposed implementation focuses on demonstrating the integration of QUIC with a virtual network interface and the secure transmission of IP packets through an encrypted tunnel.

5.4 Impact of artificial network delay on VPN throughput

Throughput represents the amount of data that can be successfully transmitted through a communication channel in a given period of time. To evaluate the throughput of the implemented VPN, the `iperf3` utility was used. The measurements were first performed over the direct connection between the two virtual machines and then repeated through the QUIC VPN tunnel using the virtual TUN addresses. The direct connection achieved a throughput of 1.25 Gbit/s, while the proposed QUIC VPN implementation reached a throughput of 4.82 Mbit/s. The lower throughput observed for the VPN is mainly a consequence of the current prototype implementation, where packets are processed individually in user space before being forwarded through the encrypted QUIC tunnel. The obtained results are presented in Figure 42 and provide a basis for comparison with other VPN solutions.

The throughput measurements indicate that the proposed QUIC VPN maintains stable performance under low and moderate network delays. As the artificial latency increases, the throughput gradually decreases due to the longer round-trip time required for packet acknowledgements and congestion control. The results also show that no abrupt throughput degradation occurs when moderate delays are introduced. Instead, the throughput decreases gradually as the delay increases, indicating that the QUIC congestion control mechanism adapts to changing network conditions while maintaining a stable communication channel.

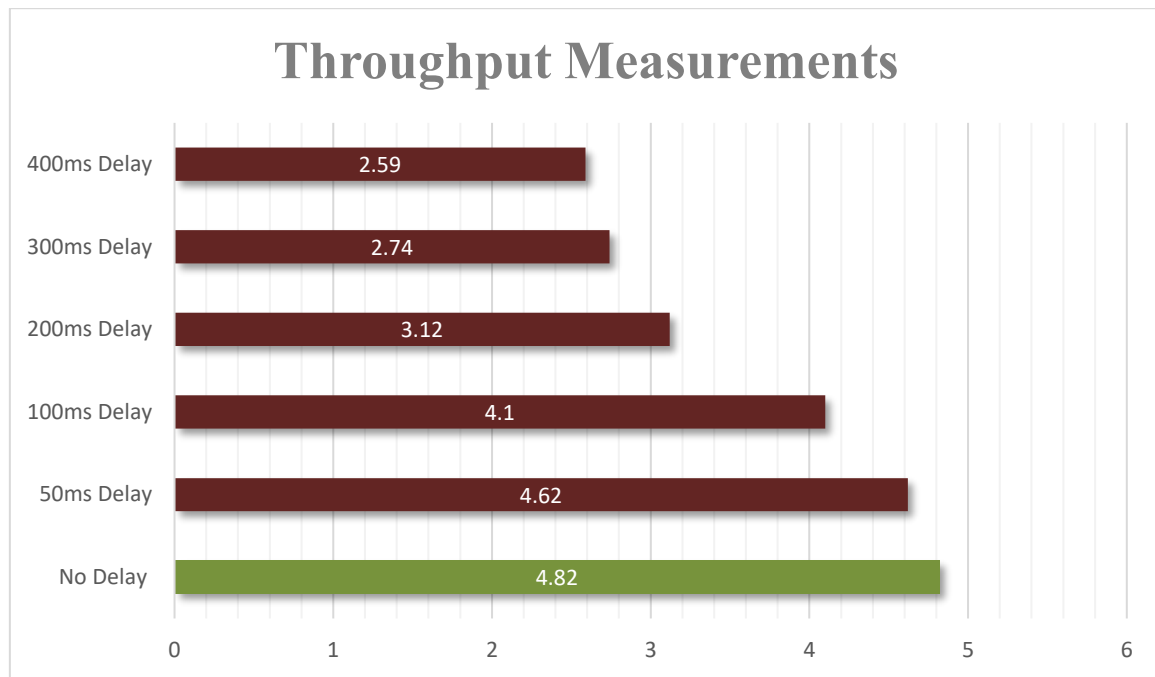


Figure 42. Throughput measurements under moderate network delays

6 Conclusions

Virtual Private Networks have become an essential technology for protecting data exchanged over public networks. As network applications continue to evolve, modern transport protocols such as QUIC offer new possibilities for improving the performance and security of VPN communications. By combining reliable transport, integrated encryption and stream multiplexing into a single protocol, QUIC provides several advantages over traditional transport protocols and represents a promising alternative for future VPN implementations. The purpose of this project was to design and implement a VPN solution based on the QUIC transport protocol and Linux TUN virtual interfaces.

The implementation was developed step by step. First, a secure QUIC connection was established between the client and the server. Once the communication channel was successfully validated, virtual TUN interfaces were integrated into both endpoints. The client captures the IP packets generated by the operating system, encapsulates them into QUIC streams and sends them to the server. After receiving the packets, the server reconstructs them, writes them to its TUN interface and forwards them to the destination network using IP forwarding and NAT. This resulted in a complete bidirectional communication flow through the VPN tunnel between the two virtual machines.

Several experiments were performed to validate the implementation. The connectivity tests confirmed that packets were successfully transmitted through the VPN tunnel without packet loss during the experiments. The captured traffic showed that the client and the server were able to exchange packets successfully through the VPN tunnel. Later, Wireshark was used to analyse the traffic generated by both the QUIC VPN and a simple TCP client-server implementation. This comparison showed the main differences between the two protocols, demonstrating how QUIC combines the transport and security mechanisms into a single protocol while TCP relies on a separate TLS layer. The connection establishment latency of the proposed implementation was evaluated and compared with OpenVPN, while throughput measurements were performed under different artificial network delays to analyse the impact of latency on the VPN performance. The implemented VPN solution was also compared with two widely used VPN technologies, WireGuard and OpenVPN. Although the developed application represents a simplified VPN prototype rather than a complete solution, the comparison showed that the proposed implementation successfully demonstrates the fundamental operating principles of a VPN while taking advantage of several features provided by QUIC, including integrated encryption, stream multiplexing and reduced connection establishment latency. The project also improved the understanding of how virtual network interfaces, QUIC communication and Linux networking mechanisms work together in a VPN solution.

The implementation demonstrates that QUIC can successfully be used as the transport protocol for a VPN solution while providing secure communication through encrypted QUIC streams. Future work may include extending the implementation with support for multiple simultaneous clients, user authentication, improved routing capabilities and performance evaluations under different network conditions. Additional experiments involving connection migration, packet loss and varying network delays could further demonstrate the advantages of QUIC in VPN applications.

During the preparation of this thesis, artificial intelligence tools were used exclusively to improve the readability and academic style of the text through paraphrasing and text reformulation.